

THE COOPER UNION FOR THE ADVANCEMENT OF SCIENCE AND ART  
ALBERT NERKEN SCHOOL OF ENGINEERING

# **Estimating Traffic Flow based on New York City Traffic Cameras with YOLOv11 for Emissions Analysis**

By

Aidan Cusa

A thesis submitted in partial fulfillment of the requirements for the degree of  
Master of Engineering

Advisor

Carl Sable

THE COOPER UNION FOR THE ADVANCEMENT OF SCIENCE AND ART

ALBERT NERKEN SCHOOL OF ENGINEERING

This thesis was prepared under the direction of Carl Sable and has received approval. It was submitted to the Dean of the School of Engineering and the full Faculty, and was approved as partial fulfillment of the requirements for the degree of Master of Engineering.

---

Barry L. Shoop, Ph.D., P.E. – Date  
Dean, Albert Nerken School of Engineering

---

Carl Sable, Ph.D. – Date  
Candidate's Thesis Advisor

# Acknowledgements

I would first like to express my sincere gratitude to Professor Carl Sable for his guidance, encouragement, and mentorship throughout this thesis process. His support and insight helped shape this work and gave me the confidence to develop it into its final form.

I would also like to thank The Cooper Union for providing the academic environment in which I was able to grow as an engineer and pursue this research. My time at Cooper Union has challenged me to think critically, work independently, and approach problems with both technical rigor and curiosity.

Finally, I am deeply grateful to my parents, Ninel and David Cusa, my sisters, Lilly, Emma, and Olivia, Julia Solines, and my extended family and friends for their constant love, patience, and support throughout this process. Their encouragement helped sustain me through the long hours, setbacks, and uncertainty that came with completing this thesis.

THE COOPER UNION FOR THE ADVANCEMENT OF SCIENCE AND ART  
ALBERT NERKEN SCHOOL OF ENGINEERING

# **Estimating Traffic Flow based on New York City Traffic Cameras with YOLOv11 for Emissions Analysis**

By

Aidan Cusa

April 30, 2026

1 – 2 PM

41 Cooper Square, Room 506

<https://cooper.zoom.us/j/93294497013>

Passcode: 846682

A thesis submitted in partial fulfillment of the requirements for the degree of  
Master of Engineering

Advisor

Carl Sable



## Abstract

Traffic-related air pollution remains a significant public health concern in dense urban environments, but traditional monitoring infrastructure cannot easily provide continuous, street-level measurements that distinguish between different types of vehicles. This thesis investigates whether existing New York City traffic-camera feeds can be used to estimate traffic flow and vehicle composition for downstream emissions analysis. To address this problem, a 1,200-image low-resolution vehicle detection dataset was constructed from 511NY and NYSDOT imagery and annotated using a 6-class structure aligned with broad EPA MOVES vehicle categories. Several object-detection architectures were evaluated, including YOLO, Faster R-CNN, RT-DETR, and Deformable DETR, with YOLOv11s providing the strongest overall performance. The baseline YOLOv11s model achieved a mAP50 of 0.804 and a mAP50-95 of 0.615 on the validation set, while the final refined model improved to a mAP50 of 0.857 and a mAP50-95 of 0.661 through targeted data refinement. On a held-out test set, the final model reached a mAP50 of 0.772 and a mAP50-95 of 0.607, while performance on a cross-camera benchmark of unseen locations dropped to a mAP50 of 0.658 and a mAP50-95 of 0.488, showing that generalization to new camera views remains a challenge. Comparison against New York City automated traffic recorder (ATR) data showed that the pipeline was better at recovering the temporal pattern of traffic flow than exact count magnitude, with the strongest agreement at Atlantic Avenue at Boerum Place, where Pearson correlation reached 0.844 and MAPE was 22.96%. Overall, the results show that public traffic-camera imagery can support practical traffic-pattern and fleet-composition analysis, while also highlighting the need for broader training data, temporal tracking, and additional operational inputs before such a system can support fully calibrated traffic counting and emissions modeling.

# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                | <b>1</b>  |
| <b>2</b> | <b>Background and Related Work</b>                 | <b>3</b>  |
| 2.1      | MOtor Vehicle Emission Simulator (MOVES) . . . . . | 3         |
| 2.2      | NYC Traffic Monitoring . . . . .                   | 4         |
| 2.2.1    | Traditional Vehicle Monitoring Methods . . . . .   | 4         |
| 2.2.2    | Traffic-Camera Based Monitoring . . . . .          | 7         |
| <b>3</b> | <b>Dataset</b>                                     | <b>8</b>  |
| 3.1      | Data Source . . . . .                              | 8         |
| 3.2      | Camera Hardware . . . . .                          | 9         |
| 3.3      | Dataset Curation . . . . .                         | 10        |
| 3.3.1    | Camera Selection . . . . .                         | 10        |
| 3.3.2    | Image Collection . . . . .                         | 13        |
| 3.4      | Data Labeling . . . . .                            | 15        |
| 3.4.1    | Annotation Class Design . . . . .                  | 16        |
| 3.4.2    | Attributes . . . . .                               | 19        |
| 3.4.3    | Unknown Vehicle Class . . . . .                    | 20        |
| 3.4.4    | Annotation Procedure . . . . .                     | 22        |
| 3.5      | Dataset Statistics . . . . .                       | 23        |
| 3.6      | Dataset Splits . . . . .                           | 23        |
| <b>4</b> | <b>Methodology</b>                                 | <b>26</b> |
| 4.1      | YOLO-Based Object Detection . . . . .              | 26        |
| 4.1.1    | Convolutional Neural Networks (CNNs) . . . . .     | 26        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 4.1.2    | Grid-Based Prediction . . . . .                               | 28        |
| 4.1.3    | Bounding Box Prediction . . . . .                             | 29        |
| 4.1.4    | Class Probability Prediction . . . . .                        | 30        |
| 4.1.5    | Post-Processing and Final Detections . . . . .                | 31        |
| 4.1.6    | YOLOv11 Architecture . . . . .                                | 32        |
| 4.2      | MOVES-based Taxonomy . . . . .                                | 33        |
| 4.3      | Handling Unknown Vehicles . . . . .                           | 34        |
| 4.4      | Performance Metrics . . . . .                                 | 36        |
| 4.4.1    | Intersection over Union . . . . .                             | 36        |
| 4.4.2    | Confusion Matrix . . . . .                                    | 37        |
| 4.4.3    | Precision-Recall Curve . . . . .                              | 38        |
| 4.4.4    | Average Precision and mAP50–95 . . . . .                      | 39        |
| 4.5      | Experimental Environment . . . . .                            | 41        |
| <b>5</b> | <b>Experimentation and Results</b>                            | <b>42</b> |
| 5.1      | YOLO-Based Detector Development and Results . . . . .         | 42        |
| 5.1.1    | Baseline YOLO Experiments . . . . .                           | 42        |
| 5.1.2    | Data-Centric Refinements . . . . .                            | 45        |
| 5.1.3    | Final YOLO Model Selection and Test-Set Evaluation . . . . .  | 52        |
| 5.2      | Comparison with Alternative Detection Architectures . . . . . | 52        |
| 5.2.1    | Faster R-CNN . . . . .                                        | 52        |
| 5.2.2    | Detection Transformer (DETR) . . . . .                        | 54        |
| 5.3      | Model Robustness and Cross-Camera Generalization . . . . .    | 56        |
| 5.3.1    | Confidence Threshold and NMS Sensitivity Analysis . . . . .   | 56        |
| 5.3.2    | Cross-Camera Generalization . . . . .                         | 59        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 5.4      | Comparison with ATR Traffic Counts . . . . .                      | 60        |
| <b>6</b> | <b>Discussion</b>                                                 | <b>64</b> |
| 6.1      | Evaluation of Detection Architectures for NYC Imagery . . . . .   | 64        |
| 6.2      | Rationale for the MOVES-Based Taxonomy . . . . .                  | 65        |
| 6.3      | Performance Gains from Targeted Data Refinement . . . . .         | 66        |
| 6.4      | Confidence Threshold Selection for Practical Inference . . . . .  | 67        |
| 6.5      | Cross-Camera Generalization and Domain Sensitivity . . . . .      | 68        |
| 6.6      | Analysis of Traffic Count Agreement . . . . .                     | 69        |
| 6.7      | Implications for Public Health and Environmental Policy . . . . . | 71        |
| <b>7</b> | <b>Limitations</b>                                                | <b>72</b> |
| 7.1      | Imaging and Dataset . . . . .                                     | 72        |
| 7.2      | Modeling and Taxonomy . . . . .                                   | 73        |
| 7.3      | Downstream Counting and Deployment . . . . .                      | 74        |
| <b>8</b> | <b>Future Work</b>                                                | <b>76</b> |
| <b>9</b> | <b>Conclusion</b>                                                 | <b>77</b> |
|          | <b>Appendix</b>                                                   | <b>79</b> |
| <b>A</b> | <b>Transformer Training Augmentation Pipeline</b>                 | <b>79</b> |
|          | <b>References</b>                                                 | <b>81</b> |

# List of Figures

|    |                                                                                          |    |
|----|------------------------------------------------------------------------------------------|----|
| 1  | Inductive loop detectors embedded in pavement. . . . .                                   | 5  |
| 2  | Pneumatic tubes used for traffic monitoring. . . . .                                     | 6  |
| 3  | Example PTZ camera models used in NYSDOT/NYC traffic monitoring infrastructure. . . . .  | 9  |
| 4  | Images returned by unavailable cameras. . . . .                                          | 10 |
| 5  | Examples of vehicle viewpoints across traffic cameras. . . . .                           | 12 |
| 6  | Map of selected CCTV cameras in NYC. . . . .                                             | 13 |
| 7  | Counts of images in the dataset by hour of day. . . . .                                  | 14 |
| 8  | Traffic camera images under different weather and lighting conditions. . . . .           | 14 |
| 9  | CVAT environment used to label images. . . . .                                           | 15 |
| 10 | Example of sedan and SUV labeling heuristics. . . . .                                    | 17 |
| 11 | Example of SUV and minivan labeling heuristics. . . . .                                  | 17 |
| 12 | Example of an attribute given to a vehicle in CVAT. . . . .                              | 20 |
| 13 | Example of ambiguous vehicles labeled as <code>unknown_vehicle</code> . . . . .          | 21 |
| 14 | Example of the YOLO pipeline. Figure adapted from [55]. . . . .                          | 26 |
| 15 | Example of convolutional neural network operations. . . . .                              | 27 |
| 16 | Conceptual grid created by the YOLO architecture. . . . .                                | 28 |
| 17 | Bounding boxes created by the YOLO architecture. . . . .                                 | 29 |
| 18 | Conceptual illustration of class predictions across the YOLO grid. . . . .               | 31 |
| 19 | Example final class-predictions and bounding boxes. . . . .                              | 32 |
| 20 | Final MOVES-based class mapping used in this study. . . . .                              | 34 |
| 21 | Example of <code>unknown_vehicle</code> masking during training set preparation. . . . . | 35 |
| 22 | Example of intersection and union for IoU . . . . .                                      | 36 |

|    |                                                                                  |    |
|----|----------------------------------------------------------------------------------|----|
| 23 | Intersection over Union calculation. . . . .                                     | 37 |
| 24 | Confusion matrix structure. . . . .                                              | 37 |
| 25 | Average precision on a precision-recall curve. . . . .                           | 40 |
| 26 | Baseline YOLO validation mAP50 and mAP50–95 over 50 epochs. . . . .              | 44 |
| 27 | YOLOv11s and YOLOv26s validation confusion matrix before refinement. . . . .     | 46 |
| 28 | YOLOv11s and YOLOv26s validation confusion matrix after refinement. . . . .      | 47 |
| 29 | Example of a hard negative from car/truck confusion mining. . . . .              | 49 |
| 30 | Example of confidence-threshold and NMS sensitivity. . . . .                     | 57 |
| 31 | Heatmaps showing performance sensitivity to confidence and NMS settings. . . . . | 58 |
| 32 | New camera imagery used for inference. . . . .                                   | 60 |
| 33 | Hourly YOLO and ATR traffic-count comparisons. . . . .                           | 63 |
| 34 | Examples of image-quality issues in traffic-camera feeds. . . . .                | 73 |

# 1 Introduction

Urban air pollution remains one of the most critical public health and environmental challenges of the twenty-first century. The World Health Organization estimates that approximately 99% of the global population breathes air containing pollutant concentrations above guideline limits, and the *State of Global Air 2025* report attributes 7.9 million deaths in 2023 to air pollution worldwide [1], [2]. Transportation is an important contributor to this burden. The IPCC Sixth Assessment Report states that the transport sector accounted for 23% of global energy-related CO<sub>2</sub> emissions in 2019, with road vehicles responsible for the majority of that total [3], [4]. In urban environments, these emissions are especially consequential because they are released close to where people live, work, and travel [5].

New York City illustrates this problem clearly. It combines a dense population, heavy roadway activity, and substantial exposure to traffic-related air pollution [6], [7]. According to the NYC Department of Health and Mental Hygiene, fine particulate matter (PM<sub>2.5</sub>) is the most harmful air pollutant for humans to breathe, and everyday car, bus, and truck traffic accounts for 14% of PM<sub>2.5</sub> emitted by NYC activities [7]. At the same time, vehicle emissions are not distributed uniformly across the urban environment. They vary by location, time of day, congestion, and, importantly, by vehicle type. Heavy vehicles, for example, contribute disproportionately to pollutants such as nitrogen oxides and particulate matter relative to light-duty passenger vehicles [8]. As a result, estimating localized transportation emissions not only requires the total traffic volume, but also information about the composition of traffic flow.

Obtaining this type of information continuously and at the street level remains difficult. Traditional traffic-monitoring methods, such as manual counts and automated traffic recorders (ATRs), remain essential for transportation planning, but they are often collected over limited durations and at limited locations [9], [10]. Likewise, fixed air-quality monitoring stations measure pollutant concentrations in the air, but they do not directly identify which vehicle classes are present at a given place and time. These limitations matter because localized emissions burdens are shaped not only by how many vehicles are present, but by what kinds of vehicles are moving through a corridor and under what traffic conditions.

Existing traffic-camera infrastructure offers one possible way to address this gap. New York City and New York State maintain large networks of public traffic cameras, and the

511NY system provides access to many live roadway images without requiring new sensing hardware [11]–[13]. These cameras were originally deployed for traffic operations and incident monitoring, and the imagery they provide is challenging for computer vision. The images from the NYC CCTV traffic cameras used in this study have a resolution of  $352 \times 240$  pixels and often contain distant vehicles, motion blur, occlusion, glare, rain, and non-ideal viewing angles. Despite the low image quality, this infrastructure offers an appealing foundation for scalable observation if vehicles can be detected and categorized reliably enough for downstream analysis.

To tackle this problem, a vehicle-detection pipeline was developed using publicly available New York City traffic-camera imagery and evaluated for emissions-oriented traffic measurement. The approach is built around a YOLOv11-based detector trained on a custom annotated dataset of low-resolution urban traffic scenes. Rather than using a generic vehicle taxonomy, the detector is trained and evaluated with a class structure tailored to the EPA’s MOtor Vehicle Emission Simulator (MOVES). This taxonomy balances what can be distinguished reliably in low-resolution imagery with what remains useful for downstream emissions analysis [14]. Ambiguous vehicles are handled explicitly through an `unknown_vehicle` label during annotation and through ignore-aware training and evaluation procedures, rather than being forced into potentially incorrect supervised classes.

The experimental results show that the final YOLOv11s detector outperformed the Faster R-CNN and DETR-style alternatives evaluated in this study, and that the most meaningful gains came from targeted data-centric refinement rather than broader tuning alone. The final model was assessed not only on a held-out test set, but also on unseen traffic cameras and through comparison with ATR hourly traffic counts. These evaluations were included because the goal of the work was not simply to maximize object-detection benchmark performance, but to determine whether traffic-camera imagery can support practical count-based workflows relevant to emissions estimation. The results suggest that low-resolution public traffic-camera imagery, despite its visual limitations, can still support useful vehicle counting and categorization when the detection task is formulated around the realities of the imagery and the needs of the downstream application. In this sense, the proposed pipeline is positioned not as a replacement for existing traffic or air-quality monitoring systems, but as a complementary framework for producing more spatially and temporally resolved information about traffic composition from infrastructure that already exists.



## 2 Background and Related Work

### 2.1 MOrtor Vehicle Emission Simulator (MOVES)

Accurately estimating the environmental impacts of transportation requires converting vehicle activity to pollutant emissions. In the United States, the primary tool for this purpose is the MOrtor Vehicle Emission Simulator (MOVES), developed and maintained by the U.S. Environmental Protection Agency (EPA). MOVES is a comprehensive emissions modeling framework used by government agencies and researchers to estimate emissions from mobile sources at local, state, and regional scales [15], [16]. According to the MOVES Technical Guidance, the program is used by state governments to “develop emission inventories for the development of state implementation plans (SIPs), transportation conformity determinations, and analyses required under the National Environmental Policy Act (NEPA)” [15].

MOVES operates by calculating baseline emission rates for various vehicle classes under specific operating conditions. To compute these values, the model requires detailed inputs, including vehicle fleet composition, age distributions, fuel properties, weather conditions, and roadway activity data [15]. By scaling these calculated emission rates with localized traffic data such as vehicle miles traveled (VMT) and traffic speeds, planners can conduct air quality assessments [17]–[19], evaluate congestion mitigation strategies [20], and model the environmental impact of infrastructure changes [21].

A critical requirement for emissions modeling within MOVES is accurate data on vehicle class distributions. The model relies on distinguishing between multiple vehicle types, as each has vastly different emission characteristics [15]. See Table 1 for the vehicle types tracked by MOVES. Currently, transportation agencies rely on traditional traffic monitoring methods to provide these class counts (see Section 2.2.1). However, these methods typically produce sparse or short-duration measurements [9]. This spatial and temporal limitation prevents the ability to capture the complete distribution of urban traffic at any point in the year, making it difficult to determine localized emissions rates.

The approach proposed in this thesis aims to bridge this data gap. By applying computer vision techniques to existing public traffic camera networks, the proposed pipeline detects and classifies vehicles to generate higher-resolution traffic composition estimates across more locations and times than short-duration manual or ATR campaigns alone. Replacing short-term samples with longer-term observations can improve the activity inputs used in

Table 1: MOVES Source Types and Highway Performance Monitoring System (HPMS) Vehicle Types [15].

| Source Type ID | Source Type                  | HPMS Vehicle Type   |
|----------------|------------------------------|---------------------|
| 11             | Motorcycle                   | Motorcycles         |
| 21             | Passenger Car                | Light Duty Vehicles |
| 31             | Passenger Truck              | Light Duty Vehicles |
| 32             | Light Commercial Truck       | Light Duty Vehicles |
| 41             | Other Buses                  | Buses               |
| 42             | Transit Bus                  | Buses               |
| 43             | School Bus                   | Buses               |
| 51             | Refuse Truck                 | Single Unit Trucks  |
| 52             | Single Unit Short-haul Truck | Single Unit Trucks  |
| 53             | Single Unit Long-haul Truck  | Single Unit Trucks  |
| 54             | Motor Home                   | Single Unit Trucks  |
| 61             | Combination Short-haul Truck | Combination Trucks  |
| 62             | Combination Long-haul Truck  | Combination Trucks  |

MOVES and support more accurate localized emissions estimates. Ultimately, this methodology is built to support the MOVES framework, bridging the gap between static monitoring practices and the dynamic reality of urban traffic.

To understand the necessity of this image-based measurement framework, it is important to examine the current standards and limitations of urban traffic data collection.

## 2.2 NYC Traffic Monitoring

### 2.2.1 Traditional Vehicle Monitoring Methods

Historically, traffic monitoring in New York City has relied on a combination of manual observation and automated counts. For intersection and signal studies, NYC DOT states that inspectors conduct manual counts of vehicles and pedestrians, typically during morning and evening rush hours, and that these counts may include the number of turning vehicles, school-hour activity, and off-peak period activity [22].

The NYC DOT also collects roadway traffic volume data using automated traffic recorders (ATRs), which measure hourly vehicle counts over several weekdays or weekend periods [22]. These devices are typically deployed during short-duration monitoring campaigns and are used to estimate traffic conditions such as peak-hour volumes and average daily traffic.

Although NYC DOT does not publicly specify the exact sensor types used for each traffic counting campaign, the Federal Highway Administration (FHWA) identifies several technologies commonly used in ATR systems depending on the duration of monitoring and installation constraints [9].

Long-term ATR sensors are primarily inductive-loop detectors embedded in the roadway surface. As shown in Figure 1, inductive loops consist of wire coils installed beneath the pavement that generate a magnetic field. When a metal vehicle passes over the loop, the vehicle alters the inductance of the loop, allowing the system to detect and count vehicles. Because these sensors are permanently installed and relatively durable, they are commonly used for long-term monitoring stations on highways and major arterial roads [9], [23].



Figure 1: Inductive loop detectors circled in yellow. Image adapted from [24].

For short-term traffic counting, which could last from several hours to several weeks, transportation agencies use temporary sensors that can be installed quickly and removed after the counting period. According to the FHWA guidelines, these sensors include pneumatic road tubes, piezoelectric strips, tape switches, and temporary inductive loop detectors placed on or within the roadway [9].

The most widely used temporary ATR sensor is the pneumatic road tube. Figure 2 shows pneumatic tubes stretched across a roadway and connected to a roadside data recorder. When a vehicle axle passes over the tube, it compresses the tube and generates a pulse of air that travels through the hose to the recorder, which logs the event as a vehicle detection. Using the timing and spacing between pulses, traffic counters can estimate vehicle counts, speed, and sometimes vehicle classification. Pneumatic tube counters are popular because they are inexpensive, portable, and easy to deploy for short-term traffic

studies. However, the tubes degrade over time due to repeated vehicle loading, their speed measurements become less reliable at very low speeds (typically below 10 miles per hour), and their accuracy decreases when monitoring multiple lanes simultaneously [9], [25].



Figure 2: Pneumatic tubes deployed on the road [26].

The NYC DOT publishes traffic volume counts on the NYC Open Data portal collected by ATRs. The data has been collected since 2007 and includes vehicle counts from more than 2,300 distinct streets across the five boroughs [10]. These records provide valuable insight into traffic volumes across the city and are widely used for transportation planning and infrastructure analysis [27].

However, these measurements represent sample counts rather than continuous monitoring. The dataset documentation notes that the number of days recorded varies by location and year, and many locations are only measured during short-duration campaigns [10], [28]. In a city as spatially and temporally dynamic as New York, where traffic conditions vary significantly by location, season, and time of day, these sampling approaches limit the temporal resolution of available traffic data.

NYC’s regional transportation reporting reflects these characteristics of traditional monitoring systems. The New York Metropolitan Transportation Council’s Hub Bound Travel program, for example, measures traffic entering and leaving the Manhattan Central Business District (defined as Manhattan south of 60th Street). The 2024 Hub Bound report indicates that the analysis relies on multiple data sources, including NYCDOT tube counts at locations such as 60th Street and the East River bridges, as well as toll transaction data from MTA Bridges and Tunnels and the Port Authority. The data they use is collected on a single weekday in the fall (2024 analysis used observations from October 16, 2024) to

estimate travel patterns entering the Central Business District [28]. While these datasets provide insights into regional traffic behavior, they still depend on periodic counts and aggregated agency data rather than continuous direct observation of traffic flows.

### **2.2.2 Traffic-Camera Based Monitoring**

In recent years, New York City has begun exploring camera-based approaches to street monitoring that can capture traffic activity continuously and across more locations. In 2023, NYC DOT launched a pilot program with the analytics company Viva to use “street activity sensors” developed by Viva to analyze street activity [29], [30]. The goal of the program was to analyze the effectiveness and safety of programs and street designs, prioritize projects for areas most in need of safety and other street improvements, and better understand how people are using the streets. The system also counts modes of travel in real-time, including cars, trucks, bicycles, scooters, buses, and pedestrians [29]. This approach offers the potential to expand traffic monitoring beyond the spatial and temporal constraints of traditional counting campaigns.

Research efforts have further demonstrated the potential of using existing traffic camera infrastructure as a large-scale sensing network. The NYU C2SMARTER Manhattan Congestion Tracker, for instance, analyzes imagery from roughly 400 of the city’s more than 900 public traffic cameras to estimate traffic density within and around the congestion pricing zone. Images are captured approximately every 30 minutes and processed to estimate activity for multiple road users, including cars, trucks, pedestrians, and bicycles [31]. Similarly, recent research by Turkcan et al. proposes a computer-vision pipeline that processes imagery from over 900 traffic cameras across New York City to analyze traffic conditions associated with congestion pricing policies [32]. These studies demonstrate how existing camera infrastructure can be leveraged to support continuous, large-scale traffic monitoring using computer vision.

This context motivates the approach presented in this thesis. While traditional methods such as manual counts and ATR-based sampling remain essential components of transportation monitoring programs, they are inherently limited in spatial coverage, duration, and labor efficiency. Camera-based systems offer the potential for continuous, scalable, and multimodal traffic observation using infrastructure that is already deployed across the city. Building on this transition, this thesis proposes a computer-vision pipeline that uses New York City traffic camera imagery to detect and classify vehicles for downstream emissions estimation. In this sense, the proposed system extends traditional

traffic monitoring practices by transforming periodic counting methods into an image-based measurement framework capable of supporting both transportation analysis and environmental assessment.

### 3 Dataset

To train and evaluate the vehicle detection pipeline, a custom dataset of real-world NYC traffic imagery was required. Rather than relying on pre-existing generic vehicle datasets (e.g., Stanford Cars [33] or BIT-Vehicle [34]), this study utilized publicly available traffic camera closed-circuit television (CCTV) feeds. This choice was motivated by the fact that the physical camera infrastructure is already installed on hundreds of streets across the five boroughs, eliminating the need for new equipment and installations. Additionally, images collected from the NYC CCTV traffic cameras have a unique resolution and viewing angle, making it difficult to generalize vehicles from other datasets to vehicles from NYC CCTV images. Furthermore, recent state legislation authorized the New York City Department of Transportation (NYC DOT) to quadruple its automated camera network, expanding to 600 equipped intersections by the end of 2026 [35]. By building on this existing and continually expanding network, this methodology ensures that the proposed vehicle detection pipeline can be deployed and scaled citywide without requiring any additional physical infrastructure installation.

#### 3.1 Data Source

The primary source of images was the 511NY portal, an official service operated in conjunction with the New York State Department of Transportation (NYSDOT), the New York City Department of Transportation (NYC DOT), and private video distribution networks such as Skyline [11], [36], [37]. The 511NY system provides real-time traffic and transit information, including access to hundreds of live traffic cameras deployed across the five boroughs [12]. According to NYSDOT engineering specifications, the primary purpose of these CCTV cameras is to provide a means to remotely monitor traffic flow on specific sections of roadway. By utilizing these live feeds, transportation agencies can actively detect incidents, analyze urban congestion, and broadcast real-time travel conditions to the public [38].

The data can be accessed through a web-based GUI or through the 511NY Developer

REST API. Developers with access to this API can query various transportation endpoints, including `GetCameras`, `GetEvents`, and `GetMessageSigns` [13]. The `GetCameras` endpoint was queried primarily as it returned metadata for a camera’s unique ID, exact coordinates (latitude and longitude), the roadway name, and a URL to fetch the current camera image.

However, querying this endpoint only returns cameras operated by NYSDOT and Skyline rather than the complete city network. This limitation exists because state and city agencies share data differently. While NYSDOT camera feeds are freely accessible through the 511NY API, the New York City Department of Transportation (NYC DOT) requires developers to sign a formal data-sharing agreement to access its network of over 900 municipal cameras [36], [39]. To ensure this research remains easily reproducible without legal barriers, the dataset was built exclusively using the unrestricted NYSDOT and Skyline feeds.

### 3.2 Camera Hardware

The physical infrastructure powering these feeds consists of industrial-grade pan-tilt-zoom (PTZ) cameras. According to NYSDOT engineering specifications, the standard hardware deployed for these remote traffic monitoring tasks includes the Pelco Esprit Model ES31PC22-2N, legacy Pelco 9760 units, and Axis Communications Q1614-E models [38], [40]. These camera models can be seen in Figure 3.



Figure 3: Example PTZ camera models used in NYSDOT/NYC traffic monitoring infrastructure. Note: camera sizes are not to scale.

While these cameras are capable of high-resolution video, the images returned by the 511NY API `GetCameras` endpoint have a resolution of  $352 \times 240$ , and are generally low-quality. Examples of such images under various conditions can be seen in Figure 8. It is possible that the images returned from the `GetCameras` endpoint are intentionally

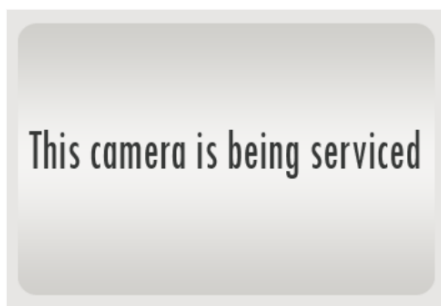
downsampled and compressed, as this reduces server bandwidth and inherently enforces privacy by making it impossible to read license plates or use facial recognition. Thus the images used in this dataset all have the dimensions  $352 \times 240$ .

### 3.3 Dataset Curation

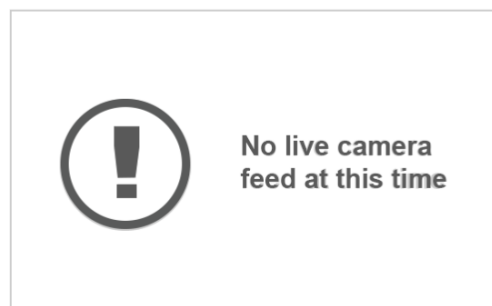
To ensure the dataset contains a robust representation of New York City traffic, the data collection targeted a diverse set of intersections from different boroughs and types of roadways (narrow commercial streets, major expressways, and neighborhood intersections).

#### 3.3.1 Camera Selection

The dataset is made up of images from 12 specific camera locations across four boroughs. These cameras were chosen based on three criteria: API availability, roadway typology, and location. First, regarding availability, requesting images from certain cameras often resulted in a continuous stream of placeholder graphics displaying “this camera is being serviced” (Figure 4a) or “no live video camera feed at this time” (Figure 4b). Only cameras with reliable uptime during the collection window were selected.



(a) Camera being serviced message.



(b) No live camera feed message.

Figure 4: Images returned by unavailable cameras.

Additionally, cameras were selected at street-level intersections rather than highways because intersections have accelerating, braking, and idling vehicles which produce higher emissions than steady highway cruising [14]. High-resolution studies further demonstrate that emissions vary sharply across short distances, with hotspots frequently forming at congested intersections that may be missed by imprecise emission estimates [8]. Also, NYC’s truck route regulations concentrate large through-trucks on designated expressways [44], while local street networks are characterized by last-mile delivery vehicles, city transit



buses, and other urban service fleets [45]. As a result, intersections better capture the vehicle composition and operating conditions that drive neighborhood-level emissions, which is the scale of interest in this study. Thus cameras were chosen in different neighborhoods in Manhattan, the Bronx, Queens, and Brooklyn. No cameras were chosen in Staten Island as they either returned images shown in Figure 4 or they were feeds of highways.

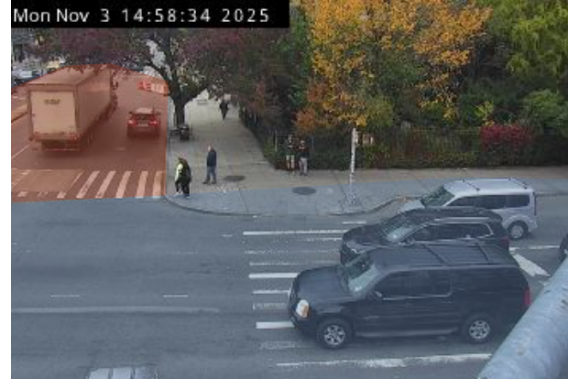
Twelve cameras that fit these criteria were then chosen based on the viewing angle of the vehicle (e.g. front view, rear view, side view, and any combination of each). This is because when developing a model to be deployed on any NYC traffic camera, it is necessary to make sure all views of different vehicle orientations are captured so the model is robust to any view of the vehicle. Figure 5 shows four different views of vehicle orientations the cameras can record. The images are colored such that vehicle regions highlighted in yellow show their front, vehicle regions highlighted in red show their rear, and vehicle regions highlighted in blue show their side. There are some cameras which only see one orientation of vehicles (Figure 5a), but there are also cameras which see two (Figures 5b and 5c) or three (Figure 5d) different vehicle orientations. The twelve chosen cameras are detailed in Table 2, showcasing their borough, street intersection, neighborhood, and vehicle views. Their physical locations within NYC are depicted in Figure 6.

Table 2: Selected CCTV Camera Locations and Viewing Orientations

| Borough   | Intersection                    | Neighborhood     | Vehicle Views     |
|-----------|---------------------------------|------------------|-------------------|
| Manhattan | 3rd Ave @ 23rd St               | Kips Bay         | Front, Rear, Side |
| Manhattan | 9th Ave @ 49th St               | Midtown          | Front, Rear       |
| Manhattan | Houston St @ Bowery St          | Lower East Side  | Front, Rear, Side |
| Manhattan | Church St @ Park Pl             | Tribeca          | Front             |
| Manhattan | Canal St @ Baxter St            | Chinatown        | Front, Rear, Side |
| Manhattan | Amsterdam Ave @ 125th St        | West Harlem      | Front, Rear       |
| Manhattan | Lenox Ave @ 135th St            | Central Harlem   | Front, Rear       |
| Bronx     | Fordham Rd @ Hughes Ave         | Belmont          | Front, Rear       |
| Queens    | Hillside Ave @ Little Neck Pkwy | Glen Oaks        | Front, Rear       |
| Brooklyn  | Atlantic Ave @ BQE              | Brooklyn Heights | Front, Side       |
| Brooklyn  | Atlantic Ave @ Vanderbilt Ave   | Prospect Heights | Front, Rear       |
| Brooklyn  | Ocean Pkwy @ Ditmas Ave         | Kensington       | Front, Rear       |



(a) Front and side view of vehicles.



(b) Side view and rear view of vehicles.



(c) Front and rear view of vehicles.



(d) Front, side, and rear view of vehicles.

Figure 5: Different views of vehicles through different traffic cameras. The images are colored such that vehicle regions highlighted in yellow show the vehicle front, vehicle regions highlighted in red show the vehicle rear, and vehicle regions highlighted in blue show the vehicle side.

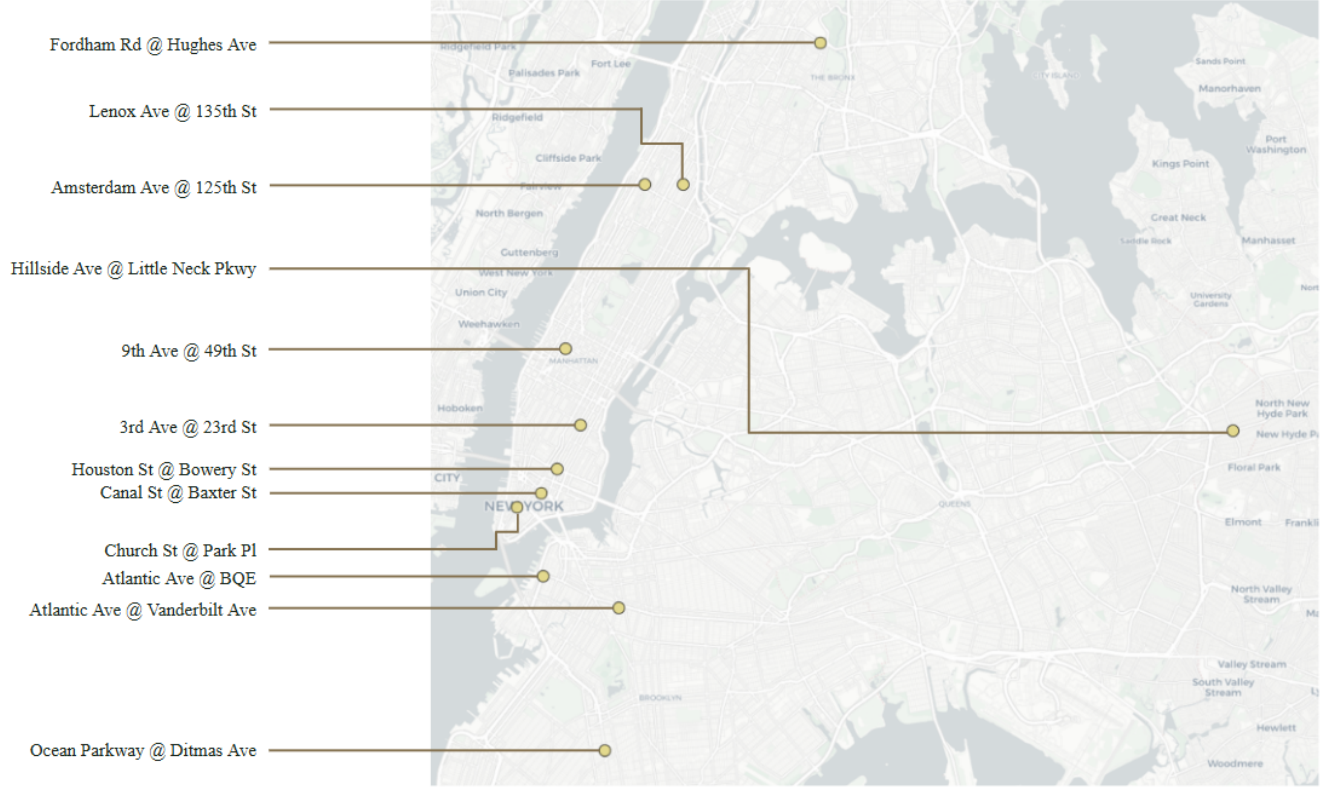


Figure 6: Map of selected CCTV cameras in NYC.

### 3.3.2 Image Collection

To automate the collection of the camera images, a Python script was developed to save images from the 12 selected cameras from the 511NY API directly. Although the API allows up to 10 requests per minute (once every six seconds) [13], and the physical cameras refresh their static feeds approximately every 15 seconds [46], a sampling rate of two minutes was chosen. This was selected to align with New York City’s traffic signal cycles, which typically range from 45 to 120 seconds depending on local traffic volumes and intersection patterns. By capturing frames every two minutes, the collection protocol inherently acts as a temporal filter, increasing the likelihood that consecutive images capture distinct vehicles rather than oversampling the same traffic scene.

Data was collected on October 1 (Wednesday), October 16 (Thursday), October 20 (Monday), and November 3 (Monday), 2025. Figure 7 shows the distribution of images in the dataset by hour of day. All images were captured between 9 AM and midnight, with most taken in the early afternoon and during the afternoon peak traffic period (3 PM to 6 PM).

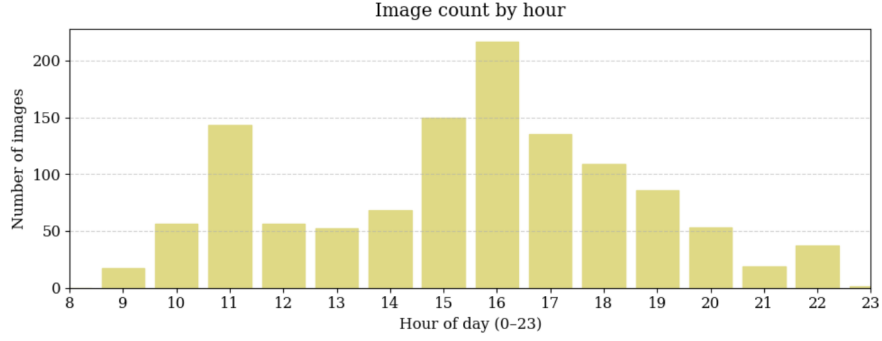


Figure 7: Counts of images in the dataset by hour of day.

Images were collected during various weather and lighting conditions so that the model could learn a robust representation of vehicles under different conditions. Examples of different conditions are shown in Figure 8.



(a) Image with sun glare.



(b) Image taken while raining.



(c) Image taken during the night.



(d) Image taken under normal sunny/cloudy conditions.

Figure 8: Examples of traffic camera images collected under different lighting and weather conditions. The dataset includes scenes affected by sun glare, rain, nighttime lighting, and more typical daytime conditions.



The final curated dataset consists of 1,200 unique images collected from 12 camera locations (100 images per camera) across four boroughs.

### 3.4 Data Labeling

After image collection, all frames were manually annotated to create the ground-truth dataset used for model training and evaluation. Because the images were collected from low-resolution traffic CCTV feeds with varying viewpoints, weather conditions, and scene complexity, the labeling process required a tool that allowed precise bounding-box annotation and consistent class management across a large image set. For this purpose, the Computer Vision Annotation Tool (CVAT) was used. The annotation environment is shown in Figure 9.

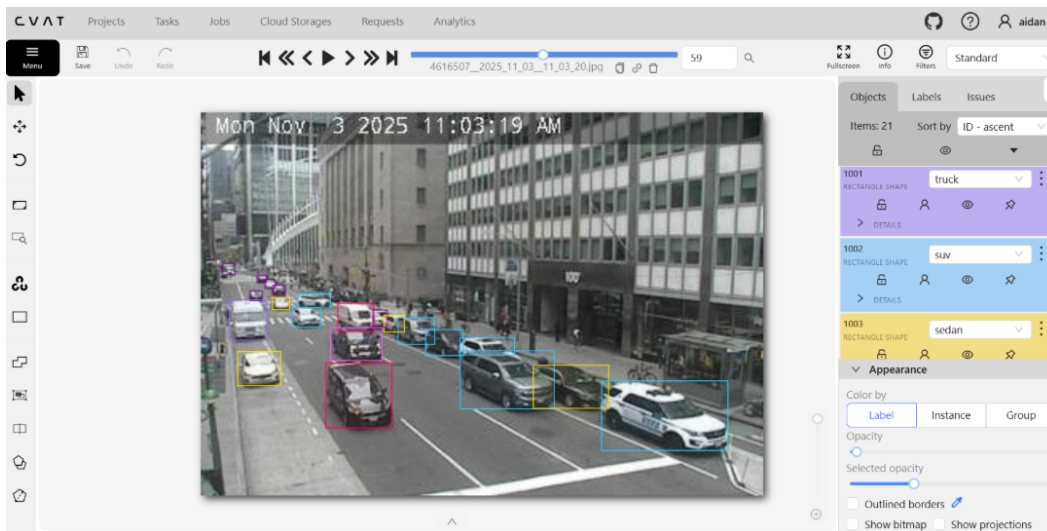


Figure 9: CVAT environment used to label images.

CVAT is a web-based annotation platform designed for computer vision datasets. It supports object detection tasks through rectangular bounding boxes and allows custom class definitions, attribute assignment, and export to multiple machine-learning formats [47]. In this study, CVAT was used to draw a bounding box around each visible vehicle and assign it a class label. Additional boolean attributes were attached when a vehicle belonged to a more specific subtype of interest, such as a taxi, police vehicle, minivan, or ambulance. This workflow allowed the dataset to preserve both a core detection taxonomy and more detailed contextual information without splitting up the main class structure.

### 3.4.1 Annotation Class Design

The vehicle classes were designed to balance two needs: visual distinguishability in low-resolution CCTV imagery and relevance to downstream emissions estimation. Thus, in total, eight classes were used: **sedan**, **suv**, **pickup\_truck**, **van**, **bus**, **truck**, **two\_wheeler**, and **unknown\_vehicle**. These labels were chosen because they represent vehicle groupings that are reasonably aligned with broader transportation and emissions categories while still remaining visually identifiable in  $352 \times 240$  traffic-camera images. In particular, vehicle types like sedans, coupes, and hatchbacks were all labeled as **sedan**, while sport utility vehicles and crossovers were both labeled as **suv**. This consolidation was intentional. Federal transportation and emissions frameworks generally operate on broader vehicle classes rather than highly specific consumer body styles. For example, the FHWA Traffic Monitoring Guide groups sedans and coupes within the passenger-car category, while EPA and DOE materials similarly distinguish broad classes such as passenger cars, passenger trucks, buses, trucks, and motorcycles for transportation and emissions analysis [48]–[50].

This grouping was also necessary because the dataset consists of low-resolution CCTV images, in which fine-grained distinctions between similar passenger body styles are often visually unreliable. In many views, differences in roofline, rear profile, ride height, and body proportions are subtle and can be obscured by distance, motion blur, occlusion, and unfavorable camera angles. As a result, labeling decisions were guided by a small set of practical visual heuristics to distinguish broader passenger-vehicle categories as consistently as possible. For example, as illustrated in Figure 10, SUVs in side or angled views were typically identified by a taller cabin profile, greater apparent ride height, and larger overall body proportions relative to nearby sedans. Sedans, by contrast, were generally lower to the ground, with a slimmer body profile and a more downward-sloping rear. Minivans, which were labeled under the **van** class and marked with a separate attribute, were usually distinguished from SUVs, when visible with sufficient clarity, by their lower stance and by the alignment of the rear sliding-door handle near the front-door handle. This distinction is shown in Figure 11.

These cues were not treated as absolute rules, but rather as consistent guidelines used to reduce labeling variability across ambiguous cases. Grouping visually similar body styles into broader categories therefore reduced annotation ambiguity and helped limit label noise, which is especially important in a manually labeled dataset where forcing uncertain distinctions can degrade downstream model performance.



(a) Sedan and SUV distinguishing heuristics.



(b) Sedan (yellow) and SUV (blue) labels used.

Figure 10: In this example, the two vehicles in the front of the image are being compared against each other. Using the colored arrows as a comparison, the white vehicle has a shorter cabin profile and a more-downward sloping rear compared to the red vehicle. Thus, the white vehicle was labeled as **sedan** and the red vehicle was labeled as **suv**. The grey vehicle in the back has a thin body profile when compared to the road (grey arrows) and a relatively short cabin profile. These are common features associated with hatchbacks, thus the vehicle was labeled as **sedan**.



(a) Minivan handles and vehicle height.



(b) SUV handles and vehicle height.

Figure 11: In this example, a vehicle with the attribute **is\_minivan** (a) and a vehicle with the label **suv** (b) are compared in the same camera position. The handles (boxed in red) on the minivan are closer together, while the handles on the SUV are farther apart. Additionally, the body of the minivan is closer to the road compared to the body of the SUV as shown with the yellow arrows.

At the same time, the annotation classes were intentionally made somewhat more descriptive than the final class structure used in the present training pipeline. Although `suv` and `pickup_truck` are ultimately grouped into the MOVES passenger-truck category during model training and downstream emissions mapping, they were annotated separately in the raw dataset to preserve semantically useful detail at the labeling stage. This decision was made primarily for annotation clarity, interpretability, and dataset flexibility. In practice, SUVs and pickup trucks are often recognizable as distinct vehicle forms in imagery even if they are later collapsed into the same broader analytical category. Retaining these labels in the annotations therefore made the dataset easier to review and also allows it to be reused for future machine-learning tasks that may require a more detailed passenger-vehicle taxonomy. This later aggregation is still consistent with EPA guidance, which places many SUVs and similar vehicles within broader light-duty truck or passenger-truck categories for emissions analysis [49], [51].

A similar logic applies to `bus` and `truck`, but in the opposite direction. In MOVES, buses are divided into categories such as other bus, transit bus, and school bus, while trucks are separated into source types such as light commercial truck, refuse truck, single-unit short-haul truck, single-unit long-haul truck, combination short-haul truck, and combination long-haul truck [50], [51]. However, these distinctions are not based on vehicle appearance alone. EPA documentation defines bus categories partly by ownership and service type. For example, transit buses are associated with public transit operations, school buses with K–12 student transport, and other buses with non-transit, non-school use [51]. Likewise, truck source types are differentiated partly by vehicle configuration, axle arrangement, haul type, and primary distance of operation rather than only visual silhouette [14].

In low-resolution  $352 \times 240$  CCTV imagery, many of these distinctions are either not visible or not reliably inferable from a single frame. A shuttle bus operated by a private company and a visually similar shuttle operated by a public transit agency may appear nearly identical in the image while belonging to different MOVES source types. Similarly, distinguishing a single-unit short-haul truck from a single-unit long-haul truck, or a refuse truck from another heavy truck under occlusion, blur, or poor viewing angle, may not be consistently possible from image evidence alone [14], [51]. For this reason, all buses were labeled under a single `bus` class and all heavier non-pickup trucks under a single `truck` class.

One exception was the addition of the `is_school` attribute for buses. Unlike other MOVES bus subcategories, school buses are often visually distinctive because of their



characteristic body shape and highly recognizable yellow color. Rather than creating a separate top-level `school_bus` class, this subtype was recorded as an attribute so that the dataset could preserve potentially useful semantic information without expanding the main detection taxonomy. This attribute-based approach maintains annotation consistency while still allowing future analyses to identify school buses when that visual distinction can be made with reasonable confidence.

Overall, this design prioritizes visually reliable labeling while preserving useful semantic detail where possible. Classes were kept separate when the distinction was clear and potentially useful for future reuse of the dataset, as in the case of `suv` and `pickup_truck`, but were collapsed when finer distinctions depended primarily on operational or contextual information that could not be reliably inferred from low-resolution imagery, as in the case of most MOVES bus and truck subcategories. The result is an annotation scheme that is intentionally image-driven for model development, even though the later emissions analysis uses a more analytically driven mapping to MOVES source types [14], [49], [51].

### 3.4.2 Attributes

Several classes were assigned boolean attributes to capture subcategories that may be useful for later analysis:

- `sedan: is_taxi, is_police`
- `suv: is_taxi, is_police`
- `van: is_taxi, is_minivan`
- `truck: is_delivery, is_firetruck, is_ambulance`
- `bus: is_school`

No additional attributes were assigned to `pickup_truck`, `two_wheeler`, or `unknown_vehicle`.

This attribute-based design was used to make the dataset more flexible without unnecessarily expanding the number of top-level detection classes. In CVAT, attributes function as additional object properties that can enrich labels without introducing a separate object category, making them well suited for subtypes that may be analytically

meaningful but are not always reliable as standalone detection classes in low-resolution imagery [52], [53]. An example of this attribute assignment process is shown in Figure 12.

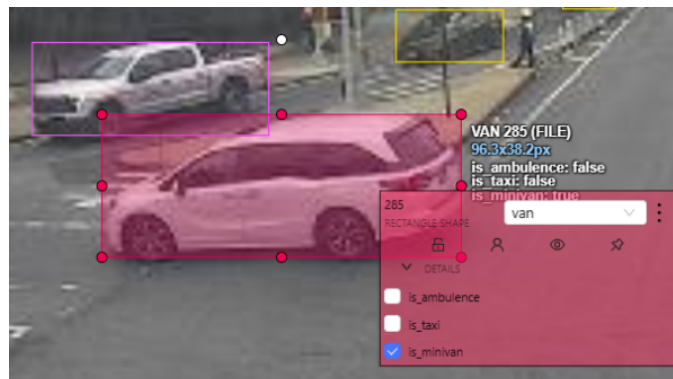


Figure 12: Example of an attribute given to a vehicle in CVAT. In this case, the **van** was given the attribute **is\_minivan** due to the labeling heuristics described in Section 3.4.1.

For this reason, the primary classes were reserved for visually robust vehicle groups, while attributes were used to preserve finer-grained semantic information when it could be identified with reasonable confidence. For example, a minivan may appear too visually similar to other vans or SUVs to justify a separate top-level class in  $352 \times 240$  traffic-camera images, yet it may still be useful to record through an attribute. Similarly, taxis, police vehicles, ambulances, fire trucks, delivery trucks, and city vehicles may be relevant for downstream policy, fleet-composition, or municipal-operations analyses, even if they are not modeled as separate classes in the present detection pipeline.

This design also improves the long-term utility of the dataset. By storing these subtypes as attributes rather than discarding them entirely, the dataset remains reusable for other machine-learning or transportation-analysis tasks, including subclass filtering, targeted error analysis, or later regrouping into alternative classes. At the same time, keeping visually ambiguous subtypes out of the main class list helps reduce label noise, which is important because noisy or inconsistent annotations can degrade model training and evaluation quality [54].

### 3.4.3 Unknown Vehicle Class

An additional class, **unknown\_vehicle**, was introduced to handle ambiguous cases where a vehicle was clearly present but could not be labeled confidently as one of the primary types. In practice, this generally occurred when the low image quality, occlusion, motion blur, extreme viewing angle, or vehicle distance made it difficult to distinguish between

visually similar classes, especially sedans and SUVs. See Figure 13 for examples.

Including the `unknown_vehicle` class served two purposes. First, it prevented uncertain objects from being forced to have incorrect labels, which would introduce label noise into the training data. Second, it allowed vehicles with uncertain class identity to remain explicitly marked as vehicles rather than being omitted from the annotations. If these regions were left unlabeled, the model could incorrectly learn that visible but ambiguous vehicles are background, and evaluation metrics could be deflated by penalizing predictions on objects whose ground-truth class was not known with sufficient confidence. This is especially important in low-resolution urban traffic imagery, where fine-grained body-style distinctions are not always visually reliable. Because the goal of the dataset was not simply to maximize the number of labeled vehicles, but to create a high-quality training set for downstream emissions-oriented classification, preserving uncertainty explicitly rather than introducing mislabeled images was preferred.



(a) Image with vehicles that are difficult to distinguish in the background (vehicles without a colorful box around them).



(b) Close-up of the background in (a) with unknown vehicles.



(c) Ambiguous vehicles from (b) are given the `unknown_vehicle` class.

Figure 13: Figure (a) shows an example frame with confidently labeled vehicles. The classes of the vehicles approaching the camera behind the crosswalk, shown in (b), are unclear because side-view features are not visible and the heuristics described in Section 3.4.1 cannot be applied reliably. These vehicles were therefore assigned the `unknown_vehicle` class, as shown in (c).

### 3.4.4 Annotation Procedure

Each image was manually reviewed in CVAT, and every visible vehicle of interest was annotated with a rectangular bounding box. Each box was drawn tightly around the visible extent of the vehicle body, after which the most appropriate class label was assigned. When applicable, additional attributes were recorded to indicate more specific vehicle roles or subtypes. All annotations were completed by the author, Aidan Cusa.

To maintain consistency, each frame was treated as an independent labeling instance. In other words, class assignments were based only on the visual evidence available in the current frame rather than how the same vehicle may have appeared in a previous or subsequent frame. This was especially important under changing lighting, occlusion, or motion conditions, where a vehicle that was clearly identifiable in one frame might be less distinguishable in another. Both moving and parked vehicles were annotated, since both constitute vehicle presence within the scene. The `two_wheeler` class was used only for motorcycles and other motorized two-wheel vehicles, and did not include bicycles or electric bicycles.

The annotation process was carried out in multiple passes. In the initial pass, clearly identifiable vehicles were labeled with bounding boxes and class assignments. A later review pass was then used to add `unknown_vehicle` instances and to correct earlier labeling issues, including missed vehicles, misclassifications, and bounding boxes that were too loose or otherwise imprecise. See Section 5.1.2 for more information regarding annotation revision.

This procedure was repeated for all 1,200 images in the dataset. Because the images were collected from multiple camera locations and viewing angles, the labeling process captured vehicles under a wide range of perspectives, including front-facing, rear-facing, side-facing, and mixed-angle views. Vehicles also appeared under varying traffic densities, illumination conditions, and levels of partial occlusion. As a result, the final annotated dataset reflects a realistic and challenging urban traffic-camera detection environment rather than a controlled benchmark setting.

### 3.5 Dataset Statistics

Across the full dataset, a total of 20,975 vehicle instances were labeled. Table 4 summarizes the number of annotated instances in each class. As expected for urban traffic-camera imagery, the largest category was **unknown\_vehicle**, reflecting the frequency of distant, occluded, or visually ambiguous vehicles in low-resolution scenes. Among the labeled vehicle classes, **suv** and **sedan** were the most common, followed by **van** and **truck**.

Table 4: Total annotated vehicle instances by class

| Class           | Count         |
|-----------------|---------------|
| sedan           | 3,357         |
| suv             | 5,010         |
| pickup_truck    | 184           |
| van             | 1,326         |
| bus             | 596           |
| truck           | 829           |
| two_wheeler     | 48            |
| unknown_vehicle | 9,625         |
| <b>Total</b>    | <b>20,975</b> |

Table 5 summarizes the number of positively assigned boolean attributes. These attributes were less frequent than the primary class labels, but they preserve useful semantic detail for future analysis, such as distinguishing taxis, police vehicles, school buses, delivery trucks, fire trucks, ambulances, and minivans. Because attributes were only assigned when the relevant subtype could be identified with reasonable confidence, their counts are substantially smaller than the corresponding parent-class totals.

Overall, the annotation design reflects a compromise between detail and reliability. The final taxonomy is sufficiently granular to distinguish major vehicle groups relevant to emissions estimation, while remaining simple enough to annotate consistently in noisy, low-resolution CCTV imagery.

### 3.6 Dataset Splits

After annotation, the full dataset was partitioned into training, validation, and test subsets using an 80/10/10 split at the image level. To reduce the risk that one subset would be dominated by only a few camera viewpoints, the split was constructed so that images from

Table 5: Positive attribute counts by parent class

| Parent Class | Attribute    | Count |
|--------------|--------------|-------|
| sedan        | is_taxi      | 17    |
| sedan        | is_police    | 19    |
| suv          | is_taxi      | 201   |
| suv          | is_police    | 30    |
| van          | is_taxi      | 161   |
| van          | is_minivan   | 301   |
| truck        | is_delivery  | 54    |
| truck        | is_firetruck | 4     |
| truck        | is_ambulance | 13    |
| bus          | is_school    | 174   |

each camera location were evenly distributed across the training, validation, and test sets. The training set was used to fit the model parameters, the validation set was used for model selection and hyperparameter tuning, and the test set was reserved for final evaluation. The final split contained 960 training images, 120 validation images, and 120 test images, corresponding to the 1,200 total annotated images in the dataset. Table 6 summarizes the number of images in each subset.

Table 6: Number of images in each dataset split

| Split        | Image Count  |
|--------------|--------------|
| Train        | 960          |
| Validation   | 120          |
| Test         | 120          |
| <b>Total</b> | <b>1,200</b> |

To ensure that the partitioning did not create a severe imbalance in object representation, the number of annotated vehicle instances in each class was also computed for each split. Table 7 shows the per-class distribution across the training, validation, and test sets. Although minor variation exists, the dominant categories remain consistent across all three subsets, with `unknown_vehicle`, `suv`, and `sedan` comprising the largest shares of the labeled objects. This indicates that the split preserved the overall class structure of the dataset while maintaining a held-out set for unbiased evaluation.

Table 7: Annotated vehicle instances by class across dataset splits. Percentages indicate each class’s share within the corresponding split.

| Class           | Train         | Validation   | Test         |
|-----------------|---------------|--------------|--------------|
| bus             | 476 (2.8%)    | 53 (2.6%)    | 67 (3.0%)    |
| pickup_truck    | 142 (0.9%)    | 25 (1.2%)    | 17 (0.8%)    |
| sedan           | 2,683 (16.1%) | 311 (15.3%)  | 363 (16.2%)  |
| suv             | 3,955 (23.7%) | 467 (22.9%)  | 588 (26.3%)  |
| truck           | 644 (3.9%)    | 120 (5.9%)   | 65 (2.9%)    |
| two_wheeler     | 37 (0.2%)     | 6 (0.3%)     | 5 (0.2%)     |
| unknown_vehicle | 7,712 (46.2%) | 928 (45.6%)  | 985 (44.0%)  |
| van             | 1,053 (6.3%)  | 125 (6.1%)   | 148 (6.6%)   |
| <b>Total</b>    | <b>16,702</b> | <b>2,035</b> | <b>2,238</b> |

The percentages show that the overall class distribution was largely preserved across the three splits, with unknown vehicles, SUVs, and sedans remaining the dominant categories in each subset. Table 8 summarizes the distribution of positively assigned attributes across the three splits. The counts of each attribute are roughly proportional to the 80/10/10 dataset split.

Table 8: Positive attribute counts across dataset splits

| Parent Class | Attribute    | Train | Validation | Test |
|--------------|--------------|-------|------------|------|
| sedan        | is_taxi      | 15    | 0          | 2    |
| sedan        | is_police    | 16    | 2          | 1    |
| suv          | is_taxi      | 166   | 25         | 10   |
| suv          | is_police    | 27    | 2          | 1    |
| van          | is_taxi      | 135   | 15         | 11   |
| van          | is_minivan   | 231   | 32         | 38   |
| truck        | is_delivery  | 50    | 4          | 0    |
| truck        | is_firetruck | 3     | 1          | 0    |
| truck        | is_ambulance | 11    | 2          | 0    |
| bus          | is_school    | 132   | 15         | 27   |

## 4 Methodology

### 4.1 YOLO-Based Object Detection

Object detection is a computer vision task that involves both localizing objects within an image and assigning them class labels. Unlike image classification, which predicts a single label for an entire image, object detection must determine both what objects are present and where they are located.

In this work, vehicle detection is performed using the YOLO family of models. As shown in Figure 14, YOLO processes an image in a single pass and directly outputs predicted bounding boxes, confidence scores, and class labels for the objects it detects. This formulation allows fast inference while maintaining strong detection performance, making YOLO well suited for traffic-camera imagery and other real-time scene-understanding tasks.

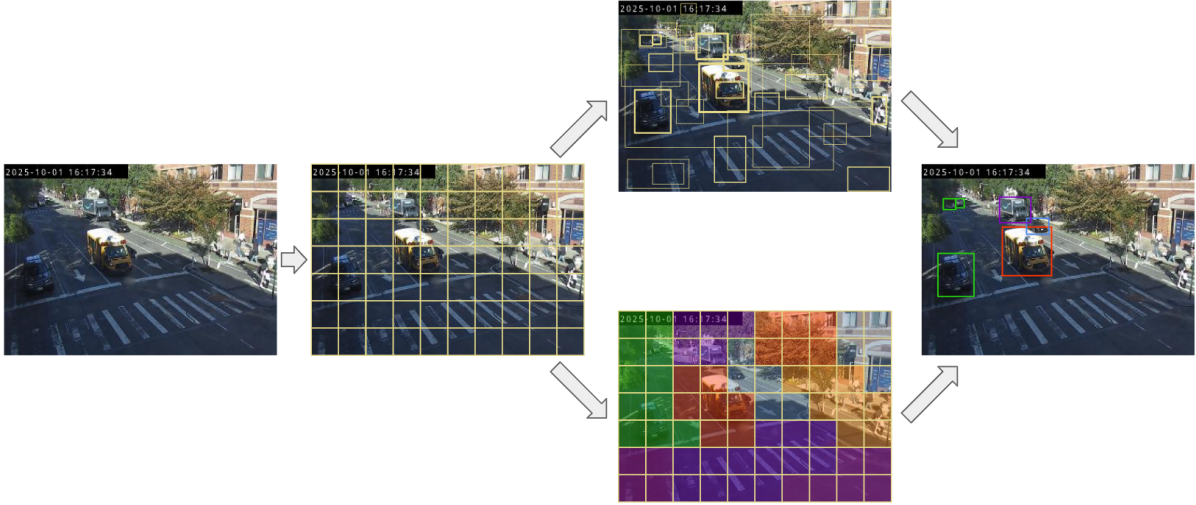


Figure 14: Example of the YOLO pipeline. Figure adapted from [55].

#### 4.1.1 Convolutional Neural Networks (CNNs)

Object detection models such as YOLO use convolutional neural networks (CNNs) to extract visual information from images [55]. Unlike traditional neural networks that treat an image as a one-dimensional vector of pixel values, CNNs operate directly on the two-dimensional structure of an image. This allows the network to learn patterns based on relationships between neighboring pixels, such as edges, textures, and shapes [56].



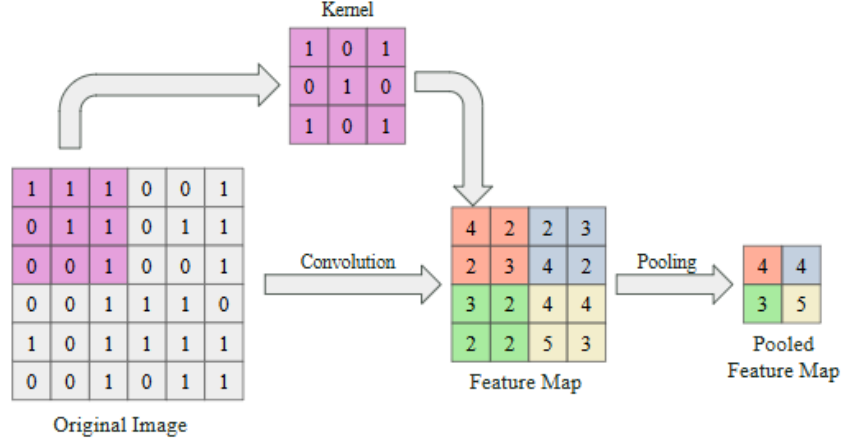


Figure 15: Example of convolutional neural network operations. A convolutional kernel slides across the input image and performs a dot product with local pixel values to produce a feature map highlighting the presence of specific patterns. In typical CNN architectures, a nonlinear activation function such as ReLU is applied after convolution. A pooling layer may then downsample the feature map while preserving the strongest activations. Figure adapted from [57].

CNNs analyze images using small filters, also called kernels, that slide across the image. At each position, the kernel performs a dot product with the underlying pixels to produce a single output value. Repeating this process across the image produces a feature map, which highlights where the pattern represented by the kernel appears in the image. During training, the network learns kernels that respond to useful visual patterns [58], [59].

Several parameters control how these operations are applied. One important parameter is the stride, which determines how far the kernel moves between successive positions. When the stride is greater than one, the resulting feature map has a lower spatial resolution than the original image. CNN architectures may also include pooling operations, which further reduce spatial resolution while preserving the strongest responses. As convolution and pooling layers are applied repeatedly, each value in a deeper feature map reflects information from a larger portion of the input image. This region is called the receptive field [59].

In Fig. 15, a simple binary image is processed using a  $3 \times 3$  kernel with stride 1, producing a feature map. This feature map is then max-pooled using a  $2 \times 2$  filter with stride 2, resulting in a  $2 \times 2$  pooled feature map containing the maximum value from each region. As convolution and pooling operations are repeated throughout the network, feature maps become smaller while their receptive fields grow larger, meaning each location summarizes

information from a larger portion of the original image.

Because each feature map location corresponds to a region of the input image, CNNs preserve a spatial relationship between feature map positions and image regions. This property allows models to generate localized predictions across the image rather than producing a single output for the entire input. Object detection models such as YOLO leverage this spatial structure to generate predictions at multiple positions in the feature map, enabling the detection of objects in different parts of the scene.

#### 4.1.2 Grid-Based Prediction

To perform object detection, the input image is conceptually divided into a  $W \times H$  grid of cells, as illustrated in Figure 16. In practice, this grid corresponds to the spatial resolution of the feature map produced by the convolutional neural network. As stated in Section 4.1.1, because convolutional layers often use strides greater than one and pooling operations reduce spatial resolution, the feature map is smaller than the original image.

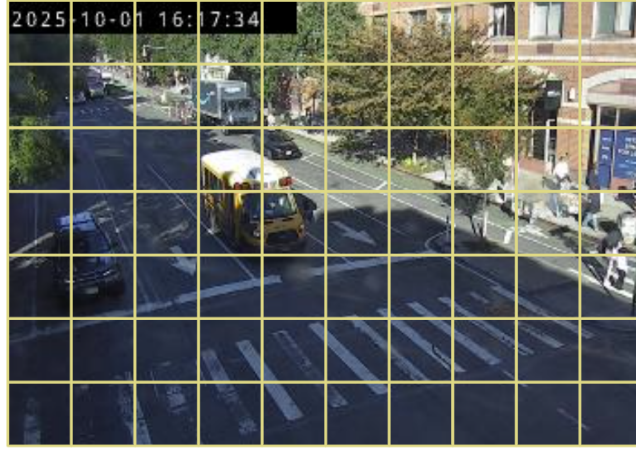


Figure 16: Conceptual grid created by the YOLO architecture. Adapted from [55].

If the network has a stride of  $s$ , the number of grid cells along the horizontal and vertical axes is approximately:

$$W_{grid} = \frac{W_{input}}{s}, \quad H_{grid} = \frac{H_{input}}{s} \quad (1)$$

where  $W_{input}$  and  $H_{input}$  are the width and height of the input image respectively. Each grid location therefore corresponds to a region of the original image and serves as a prediction point. YOLO uses these spatial locations to produce candidate bounding boxes,

objectness scores, and class probabilities for objects whose centers fall within the corresponding image region. This grid-based formulation enables the model to perform dense prediction across the image, allowing multiple objects to be detected simultaneously within a single forward pass of the network [55].

### 4.1.3 Bounding Box Prediction

For each grid cell, the CNN predicts  $B$  candidate bounding boxes along with confidence scores and class probabilities. Each predicted bounding box is parameterized by five values:

$$(x, y, w, h, c) \tag{2}$$

where:

- $x, y$  represent the center coordinates of the bounding box relative to the grid cell,
- $w, h$  represent the width and height of the bounding box relative to the image dimensions,
- $c$  represents the confidence score associated with the prediction.

These parameters jointly represent the location and size of the predicted object within the image. The bounding box predictions are conceptually shown in Figure 17.

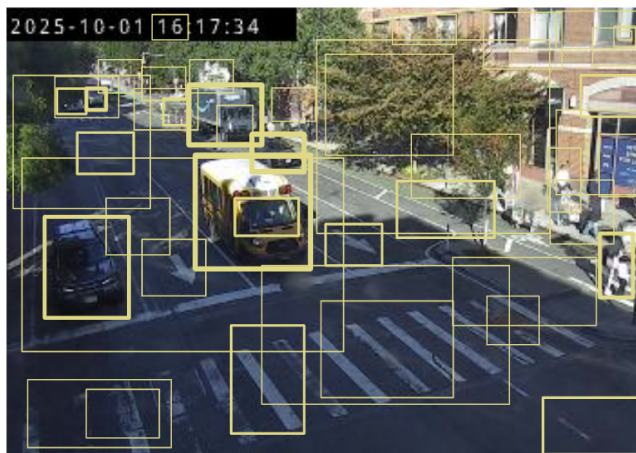


Figure 17: Bounding boxes created by the YOLO architecture. The thickness of each box corresponds to the model’s confidence score. The higher the confidence, the thicker the box. Adapted from [55].

The confidence score represents the model’s belief that a predicted bounding box contains an object and is defined as:

$$c = P(\text{object}) \times \text{IoU}_{\text{pred}}^{\text{truth}} \quad (3)$$

where  $P(\text{object})$  is the probability that an object exists in the predicted region and  $\text{IoU}_{\text{pred}}^{\text{truth}}$  is the Intersection over Union (IoU) between the predicted bounding box and the ground truth bounding box [55]. See Section 4.4.1 for more details regarding IoU. This formulation allows the model to simultaneously capture both object presence and localization accuracy in a single prediction score.

#### 4.1.4 Class Probability Prediction

In addition to predicting bounding boxes, each grid cell also produces a vector of conditional class probabilities representing the likelihood that the detected object belongs to each category. These probabilities are predicted independently of the bounding box geometry and are conditioned on the presence of an object within the grid cell.

If the model is trained to detect  $C$  object classes, each grid cell predicts:

$$P(\text{Class}_i \mid \text{object}) \quad (4)$$

for  $i = 1, 2, \dots, C$ . During inference, the final class-specific confidence score for a predicted bounding box is computed by combining the conditional class probability with the confidence score of that box  $c$ . Then,

$$\text{Score}_i = c \times P(\text{Class}_i \mid \text{object}) \quad (5)$$

This gives a class-specific confidence score for each predicted bounding box, reflecting both the likelihood that an object is present and the probability that the object belongs to a particular class. Conceptually, these class predictions are produced across the spatial grid of the image. Figure 18 illustrates this idea, where the colors indicate the dominant predicted class at different grid locations rather than the final post-processed detections [55].

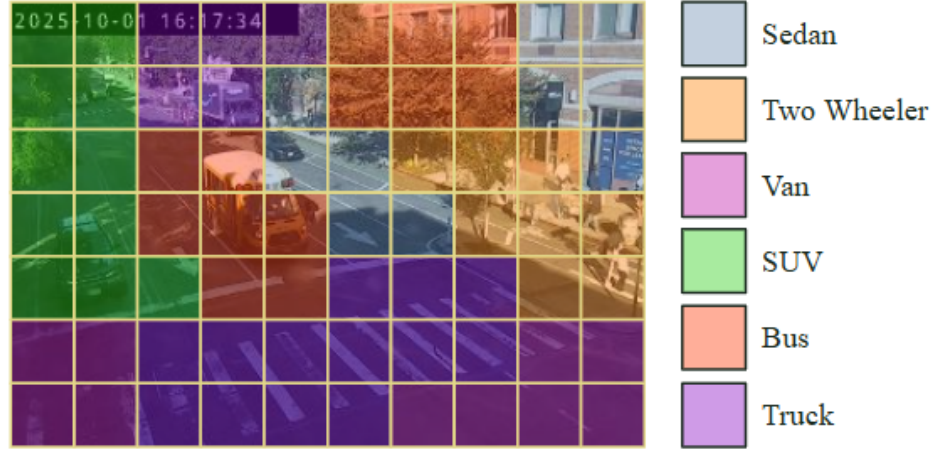


Figure 18: Conceptual illustration of class predictions across the YOLO grid. Each colored region indicates the dominant predicted class at that spatial location, based on the highest class-specific score. The figure is intended to show how class predictions are distributed across the image and does not represent the final post-processed detections. Adapted from [55].

#### 4.1.5 Post-Processing and Final Detections

Because each grid cell may contain multiple candidate bounding boxes, the raw output of the network often contains several overlapping predictions that correspond to the same object. To produce the final detections, a post-processing step known as non-maximum suppression (NMS) is applied. NMS operates by:

1. Ranking all predicted bounding boxes according to their confidence scores.
2. Selecting the highest-scoring prediction.
3. Removing any remaining predictions whose overlap with the selected box exceeds a predefined IoU threshold.
4. Repeating this process until only high-confidence predictions remain.

This procedure removes redundant detections while preserving the most confident prediction for each object in the scene. The resulting bounding boxes are the final object detections, which include the predicted object class, bounding box coordinates, and associated confidence score, as shown in Figure 19. For further information regarding the YOLO architecture and training procedure, please refer to the original work by Redmon et al. [55].



Figure 19: Example final class-predictions and bounding boxes. Adapted from [55].

#### 4.1.6 YOLOv11 Architecture

This study evaluated several YOLO variants, including YOLOv8, YOLOv11, and YOLOv26. Because YOLOv11 was ultimately selected as the strongest-performing model in the later experiments, its architecture is described here in more detail. Like the other YOLO variants considered in this work, YOLOv11 follows the general detection pipeline described in the previous sections, in which bounding boxes, object confidence scores, and class probabilities are predicted directly from convolutional feature maps in a single pass through the network. YOLOv11 is a recent member of the YOLO family that builds on earlier YOLO detectors with architectural refinements aimed at improving detection accuracy and computational efficiency.

Like many modern object detectors, YOLOv11 uses a backbone-neck-head architecture. The backbone is a deep convolutional network that extracts visual features from the input image. These features are then passed to a neck module, which combines information across multiple spatial scales so that objects of different sizes can be recognized more effectively. Finally, the detection head outputs bounding box coordinates, object confidence scores, and class probability predictions at multiple spatial locations in the feature maps [60].

Compared with earlier YOLO models, YOLOv11 introduces several refinements intended to improve both accuracy and efficiency. One important component is the C3k2 module, an enhanced convolutional block designed to improve feature extraction while reducing computational cost through more efficient reuse of intermediate feature representations. YOLOv11 also uses PAN/FPN-style neck layers to improve feature aggregation across multiple resolutions, which helps the detector handle objects at different spatial scales. In

addition, refinements to the detection head improve bounding box localization and class prediction while preserving the fast inference speed associated with YOLO-based detectors. Together, these changes make YOLOv11 a strong and computationally efficient detector for real-time applications [60].

In this study, YOLOv11 was selected for the final detector because it achieved the best empirical performance on the vehicle detection dataset while maintaining inference efficiency suitable for large-scale traffic-camera imagery. Additional details on the YOLOv11 architecture and training procedure are available in the Ultralytics documentation [61] and in the work of Khanam and Hussain [60].

## 4.2 MOVES-based Taxonomy

For the experiments reported in this chapter, model training and evaluation were conducted using a final 6-class taxonomy: `passenger_car`, `passenger_truck`, `light_commercial_truck`, `bus`, `truck`, and `motorcycle`. This label set was selected to provide a practical balance between compatibility with downstream MOVES-based emissions analysis and what could be distinguished reliably from low-resolution traffic-camera imagery. Rather than reproducing the full MOVES source-type framework, the taxonomy preserves broad categories relevant to emissions modeling while reducing visual distinctions that are difficult to learn consistently.

To achieve this balance, fine-grained vehicle labels were grouped accordingly. Sedan instances were mapped to `passenger_car`, while SUV instances, `pickup_truck` instances, and `van` instances with the attribute `is_minivan` were grouped into `passenger_truck`. The EPA notes that it is difficult to visually determine whether a vehicle should be treated as a passenger car or a light-duty truck, but classifies many SUVs and minivans as light-duty trucks rather than passenger cars [49].

The remaining van instances were mapped to `light_commercial_truck`. This separation is useful from a computer-vision standpoint, as commercial vans tend to have more consistent shapes and usage patterns than SUVs and pickup trucks. It also aligns with MOVES technical guidance, which treats passenger trucks and light commercial trucks as separate source types [14]. Additionally, since the original `two_wheeler` class only contained motorcycles, it was remapped directly to `motorcycle`.

Finally, `bus` and `truck` were preserved as independent classes because they are

operationally different, visually distinct, and treated as separate categories within MOVES. Within the bus category, however, school buses were not modeled as a separate detector class, even though MOVES distinguishes other buses, transit buses, and school buses as separate source types. Instead, school buses were retained as an annotation attribute and grouped into the broader bus class. This is because the primary objective of this study is to identify the broad vehicle categories most relevant to downstream emissions-oriented analysis rather than to infer every possible subtype within each category. As such, the bus attribute, `is_school`, was preserved as useful metadata and a potential avenue for future refinement, but it was not treated as essential to the core detector taxonomy. The final mapping between the dataset taxonomy and the MOVES taxonomy is shown in Figure 20. This MOVES-based taxonomy was used for the remainder of the study as the common label mapping for the dataset and for all subsequent training, testing, and evaluation.

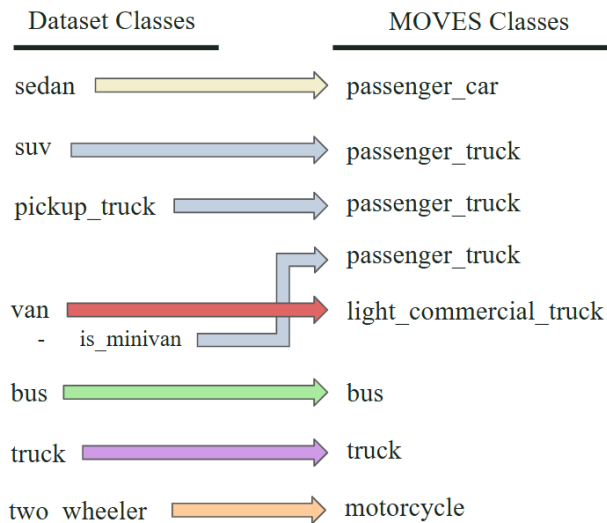


Figure 20: Final MOVES-based class mapping used in this study. Dataset classes on the left were mapped to the MOVES-aligned detector classes on the right. Arrows of the same color indicate the corresponding class mapping, with van instances labeled `is_minivan` grouped into `passenger_truck` and the remaining van instances mapped to `light_commercial_truck`.

### 4.3 Handling Unknown Vehicles

As described in Section 3.4.3, some vehicles were annotated as `unknown_vehicle` when they could be recognized as vehicles but could not be assigned confidently to one of the six supervised classes. In the detector pipeline, these regions were not treated as a seventh learnable class. Instead, they were handled as ignore regions during both dataset



preparation and evaluation.

During preparation of the model training splits, regions that were labeled `unknown_vehicle` during dataset annotation were masked in the images using a neutral fill value of `RGB(114,114,114)`. This meant that the detector was trained only on the MOVES-based target classes, while ambiguous vehicles were masked so they would not be forced into an incorrect label or treated as background even though they were clearly vehicles. Because this masking occurred during dataset generation, it was applied before any augmentation pipeline. See Figure 21 for an example of a training image before and after masking.



(a) Training image with the `unknown_vehicle` annotated regions outlined in purple.



(b) Same training image after masking regions annotated as `unknown_vehicle` with the neutral fill value.

Figure 21: Example of `unknown_vehicle` masking during training set preparation. Subfigure (a) shows the `unknown_vehicle` regions outlined in purple for reference, while subfigure (b) shows the same image after the `unknown_vehicle` regions were masked with the neutral fill value used for training.

During validation and later test-time evaluation, the images were not modified. Instead, a custom ignore-aware evaluation procedure was used. The `unknown_vehicle` boxes were reconstructed directly from the original CVAT XML annotations, and any prediction overlapping one of these regions above a set IoU threshold was removed before scoring. The remaining predictions were then matched against the MOVES-based classes to compute mAP50 and mAP50-95. This ensured that detections falling on visually ambiguous vehicles were not counted as false positives, while preserving the standard inference settings for confidence threshold, NMS IoU threshold, and maximum detections. This handling of unknown-vehicle regions was applied consistently across model architectures during training, validation, and test evaluation.

## 4.4 Performance Metrics

To determine whether computer vision can support emissions-oriented traffic measurement in New York City, the vehicle detection pipeline must be rigorously evaluated. Unlike standard image classification, object detection requires assessing two distinct tasks simultaneously: accurately localizing a vehicle within a complex urban environment and correctly classifying its emissions profile category (e.g., distinguishing a truck from a bus). This subsection details the metrics selected to quantify the model’s performance. It begins by establishing the spatial evaluation criteria, builds toward aggregate threshold-agnostic metrics like mean average precision (mAP50–95), and concludes by justifying these choices against traditional classification metrics that fall short in dynamic spatial contexts.

### 4.4.1 Intersection over Union

Before assessing the classification accuracy of the model, it is necessary to quantify the spatial accuracy of the predicted bounding boxes. This is achieved using the Intersection over Union (IoU) metric. IoU measures the overlap between a predicted bounding box and a ground-truth bounding box. It is calculated as the area of their intersection divided by the area of their union. Figure 22 illustrates the intersection and union regions for an example prediction, while Figure 23 shows the corresponding IoU calculation.

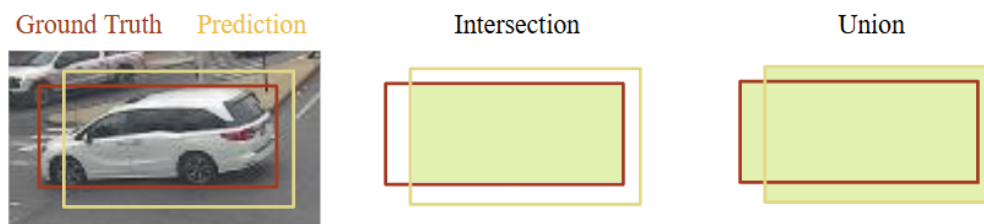


Figure 22: Example of the intersection and union between a ground-truth bounding box (red) and a predicted bounding box (gold). The figures show the original overlap, the intersection area, and the union area used in the IoU calculation.

An IoU score ranges from 0 (no overlap) to 1 (perfect alignment). By establishing an IoU threshold (e.g., 0.50), we define the strictness required for a bounding box prediction to be considered valid.

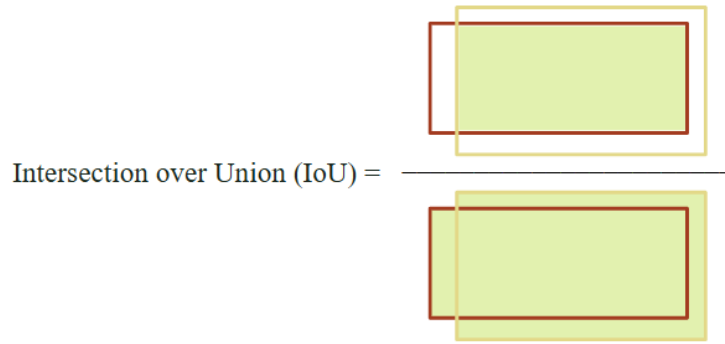


Figure 23: Intersection over Union calculation.

#### 4.4.2 Confusion Matrix

|                  |          | Ground Truth Labels |                     |
|------------------|----------|---------------------|---------------------|
|                  |          | Positive            | Negative            |
| Predicted Labels | Positive | True Positive (TP)  | False Positive (FP) |
|                  | Negative | False Negative (FN) | True Negative (TN)  |

Figure 24: Confusion matrix structure.

With an IoU threshold in place, the standard confusion-matrix categories shown in Figure 24 can also be applied to object detection. Unlike image classification, where each image is assigned a single label, object detection must evaluate both what the object is and where it is located. Because predicted bounding boxes will rarely match the annotated box exactly, the IoU threshold provides a practical way to decide whether a detection is close enough to count as correct. In this way, the IoU threshold converts box overlap into the discrete categories needed for evaluation, such as true positives, false positives, and false negatives. Within the context of this project, the entries of the confusion matrix correspond to:

- **True Positive (TP):** A detection that matches a human-annotated ground truth

box of the correct vehicle class with an  $\text{IoU} \geq$  the defined threshold. For example, the model successfully draws a bounding box around a sedan and correctly classifies it as a passenger car with an IoU that exceeds the threshold.

- **False Positive (FP):** A predicted box that does not satisfy the matching criteria for a correct detection. In this study, false positives generally occur in three ways:
  1. *Misclassification:* The predicted box overlaps a ground-truth object sufficiently, but the assigned class label is incorrect.
  2. *Poor Localization:* The predicted class is correct, but the bounding box does not overlap the ground-truth box enough to satisfy the required IoU threshold.
  3. *Background/Duplicate:* The predicted box either does not correspond to any ground-truth object or duplicates another prediction for the same object.
- **False Negative (FN):** A ground-truth vehicle that is not successfully matched by any prediction. This can occur when the model fails to detect the vehicle entirely, assigns it the wrong class, or predicts a bounding box that does not overlap sufficiently to meet the IoU threshold.
- **True Negative (TN):** Conceptually, a true negative represents every possible empty region in an image (such as patches of bare asphalt, sidewalks, buildings, or sky) where the model correctly predicted that *no* vehicle was present. Because the number of empty background regions in a dense urban scene is effectively infinite, True Negatives mathematically distort traditional evaluation metrics and are therefore excluded from object detection performance calculations.

#### 4.4.3 Precision-Recall Curve

Using the  $TP$ ,  $FP$ , and  $FN$  counts defined above, we can calculate precision and recall to evaluate the pipeline’s ability to detect vehicles reliably in urban traffic settings.

- **Precision** measures the reliability of the model’s vehicle predictions.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{6}$$

In the context of this project, out of all the bounding boxes the model predicted for a certain class (e.g. claiming to detect 50 trucks), what fraction were correct? A high

precision score ensures the model is not predicting false “ghost” vehicles or misclassifying vehicle types.

- **Recall** measures the model’s ability to find vehicles in the frame.

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

Out of all the actual ground truth objects present, what fraction did the model successfully detect? A high recall score ensures emission sources are not undercounted in the final analysis.

Modern object detectors like YOLO output a confidence score with each predicted bounding box, creating a trade-off between precision and recall. Lowering the confidence threshold allows the model to predict more vehicles (increasing recall), but it simultaneously increases the risk of capturing background noise as valid vehicles (false positives), thereby lowering precision. By calculating precision and recall at every possible confidence threshold and plotting the results, we generate the precision-recall (PR) curve. The PR curve illustrates the model’s overarching stability and allows us to visualize the exact penalty in precision when we tune the model to capture a higher volume of traffic.

#### 4.4.4 Average Precision and mAP50–95

To quantify the trade-off in the PR curve into a single scalar value, we use the average precision (AP). AP represents the total area under the PR curve, depicted in Figure 25. It represents the model’s reliability across every possible confidence threshold for a single specific vehicle class (e.g., evaluating solely the model’s ability to isolate trucks from the rest of the traffic flow). AP is calculated as follows:

$$AP = \int_0^1 p(r)dr \quad (8)$$

where  $p(r)$  is the precision at recall  $r$ . By calculating the area under the curve, AP penalizes models that suffer severe drops in precision when attempting to retrieve the hardest-to-find vehicles (such as heavily occluded trucks).

To evaluate the model across each of the MOVES-based classes, the individual AP scores for all distinct classes are averaged to produce the mean average precision (mAP):

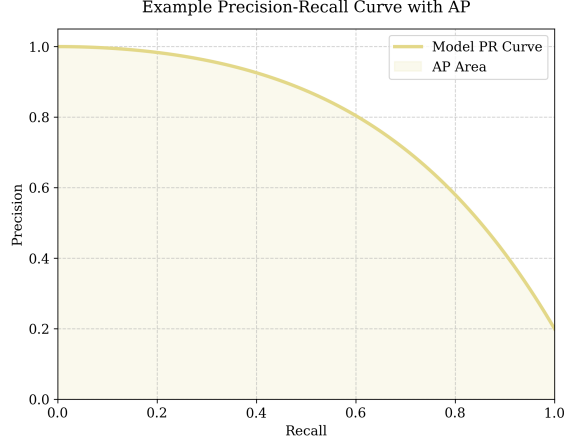


Figure 25: Average precision on a precision-recall curve.

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (9)$$

where  $N$  is the total number of classes. Because the mAP score still depends on the spatial overlap threshold, standard evaluation protocols report performance at different Intersection over Union (IoU) thresholds. A common baseline is mAP50 (AP calculated at an IoU threshold of 0.50), which tolerates moderate bounding box looseness. However, modern object detection systems evaluate performance using the protocol established by the Microsoft Common Objects in Context (MS COCO) dataset [62]. The COCO-style mAP (often denoted as mAP50–95) averages the mAP across ten distinct IoU thresholds ranging from 0.50 to 0.95 with a step size of 0.05:

$$mAP_{COCO} = \frac{1}{10} \sum_{t \in T} mAP_t \quad (10)$$

where  $T = \{0.50, 0.55, 0.60, \dots, 0.95\}$  and  $mAP_t$  is the mean average precision evaluated at IoU threshold  $t$ .

The primary objective of this thesis is accurate vehicle detection and classification for emissions-oriented traffic analysis, which makes mAP50 an intuitive metric because it measures whether a vehicle was detected with moderate spatial overlap. However, this study reports both mAP50 and mAP50–95, and uses mAP50–95 as the primary performance indicator for several reasons:

- First, mAP50–95 evaluates detection performance across multiple IoU thresholds

from 0.50 to 0.95 rather than at a single cutoff, providing a more complete measure of localization quality.

- Second, localization accuracy is especially important in low-resolution urban traffic imagery, where vehicles are often small, partially occluded, and closely spaced. In these conditions, a loose bounding box may still count as correct under mAP50 while failing to isolate the vehicle precisely.
- Third, in congested scenes, detections that overlap multiple nearby vehicles can appear acceptable at  $\text{IoU} = 0.50$  even when they are too coarse for reliable counting and class-specific analysis. Using mAP50–95 helps penalize these cases more appropriately.
- Finally, mAP50–95 is the standard summary metric in modern object detection literature, making the results easier to compare with prior work [62], [63].

The mAP50–95 metric therefore provides a more rigorous measure of detector quality than mAP50 in this study because it rewards models that not only detect vehicles, but also localize them precisely in dense urban traffic scenes.

## 4.5 Experimental Environment

All detector training and evaluation experiments were conducted in Python using Google Colab with an NVIDIA L4 GPU. YOLO models were implemented and trained using the Ultralytics library, while alternative detector models were implemented in PyTorch. Data augmentation was applied using the Albumentations library where relevant. Experiments were run with a fixed seed of 42 to improve consistency across runs.

## 5 Experimentation and Results

Vehicle detection experiments in this study were centered on a 6-class taxonomy designed to support emissions-oriented analysis. Several preliminary configurations were explored during early development, but those runs were primarily used to understand the difficulty of the task and refine the problem formulation. Thus, the results presented in this section focus on experiments conducted under the final 6-class setting.

First, YOLO-based experiments are presented, including comparisons across model variants and refinements through data-centric strategies such as hard negative mining and dataset refinement. Alternative detectors, including DETR-based models and Faster R-CNN, were examined to compare architectures within the project’s context.

After identifying the strongest detector, additional experiments evaluate its robustness through threshold and NMS sensitivity analysis, cross-camera generalization, and agreement between predicted vehicle counts and reference ATR or manual counts. Together, these experiments assess the practical suitability of the final system for traffic-monitoring applications.

### 5.1 YOLO-Based Detector Development and Results

#### 5.1.1 Baseline YOLO Experiments

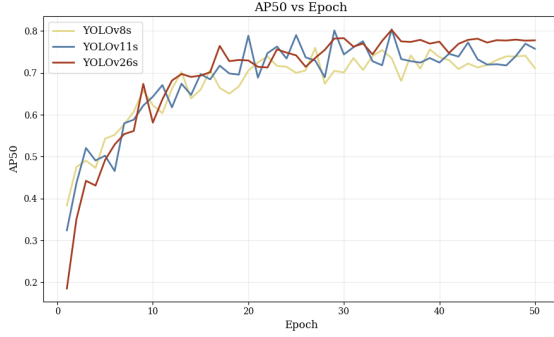
The first YOLO experiments using the MOVES-based taxonomy were conducted by fine-tuning baseline YOLO models on the training split and validating them on the validation split defined in Section 3.6. Each model was trained to predict the six final vehicle classes — passenger car, passenger truck, light commercial truck, bus, truck, and motorcycle — directly from traffic-camera images. The purpose of this stage was to assess whether YOLO could learn the 6-class formulation effectively and provide a strong starting point for later refinement.

To evaluate CNN-based one-stage detectors in a controlled and comparable way, this study focused on YOLOv8, YOLOv11, and YOLOv26, which represent three distinct generations of the Ultralytics YOLO family. YOLOv8 (2023) was selected as a strong and widely adopted baseline, since it introduced the anchor-free split head and was designed as a general-purpose real-time detector with a favorable speed-accuracy tradeoff [64], [65]. YOLOv11 (2024) was included because it is a newer Ultralytics detector that builds on

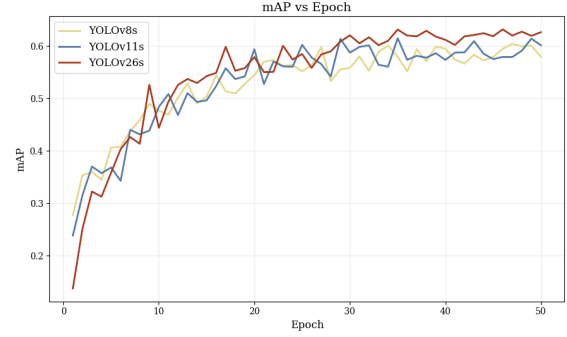


earlier YOLO models with improved feature extraction, training efficiency, and parameter efficiency, making it a natural candidate for higher accuracy under the same general framework [61], [65]. As of April 2026, YOLOv26 is Ultralytics’ most recent YOLO model, tested as an edge-oriented variant designed for simpler deployment, with end-to-end NMS-free inference and targeted improvements for efficiency and small-object recognition [65], [66]. Evaluating these three models therefore allowed comparison across a consistent detector family while covering a useful range of design priorities: a mature baseline (YOLOv8), a newer accuracy-focused refinement (YOLOv11), and a deployment-oriented architecture with simplified inference (YOLOv26). This made them well suited for the present task, which required not only accurate vehicle detection but also practical applicability to traffic-monitoring and counting workflows. To enable a more controlled comparison across YOLO generations, all experiments in this subsection used the small (S) variant, which provided a consistent mid-scale tradeoff between model capacity, compute, and efficiency [63].

To establish baseline performance, each YOLO variant was fine-tuned under the same general training configuration. All baseline models were trained for 50 epochs with a batch size of 8, since the validation mAP50–95 curves indicated that performance had largely plateaued by that point. Unless otherwise noted, the same default augmentation pipeline was used across all YOLO variants so that observed differences would primarily reflect architectural differences rather than changes in training conditions. During training and evaluation, images were resized to  $768 \times 1280$ , and bounding-box annotations were converted from the original CVAT XML format into normalized YOLO label files under the final 6-class mapping. Instances labeled `unknown_vehicle` were not treated as a supervised detector class and were instead handled according to the ignore-region procedure described in Section 4.3.



(a) Validation mAP50 versus training epoch for the baseline YOLOv8s, YOLOv11s, and YOLOv26s models.



(b) Validation mAP50–95 versus training epoch for the baseline YOLOv8s, YOLOv11s, and YOLOv26s models.

Figure 26: Validation mAP50 and mAP50–95 over 50 epochs for the baseline YOLOv8s, YOLOv11s, and YOLOv26s models. All three models showed rapid early improvement followed by a gradual plateau, with YOLOv26s achieving the strongest baseline mAP50–95 and YOLOv11s achieving the strongest baseline mAP50.

Table 9: Baseline validation performance of the three YOLO model variants under the final 6-class formulation reported at the checkpoint with the best validation mAP50–95 for each model. The top score for each metric is in bold.

| Model Checkpoint | Epochs | mAP50        | mAP50–95     |
|------------------|--------|--------------|--------------|
| YOLOv8s          | 50     | 0.750        | 0.606        |
| YOLOv11s         | 50     | <b>0.804</b> | 0.615        |
| YOLOv26s         | 50     | 0.802        | <b>0.632</b> |

As shown in Table 9 and Figure 26, all three baseline YOLO variants converged within 50 epochs on the validation set under the MOVES-based taxonomy. YOLOv8s achieved a mAP50 of 0.750 and a mAP50–95 of 0.606, while YOLOv11s improved this to a mAP50 of 0.804 and a mAP50–95 of 0.615. YOLOv26s produced the highest baseline mAP50–95 value of 0.632, while remaining close to YOLOv11s in mAP50 with a value of 0.802. Since YOLOv11s and YOLOv26s represented the strongest baseline models in terms of mAP50 and mAP50–95, respectively, both were carried forward into the subsequent refinement experiments. The next stage of experiments investigated whether these baseline results could be further improved through data-centric refinement strategies.

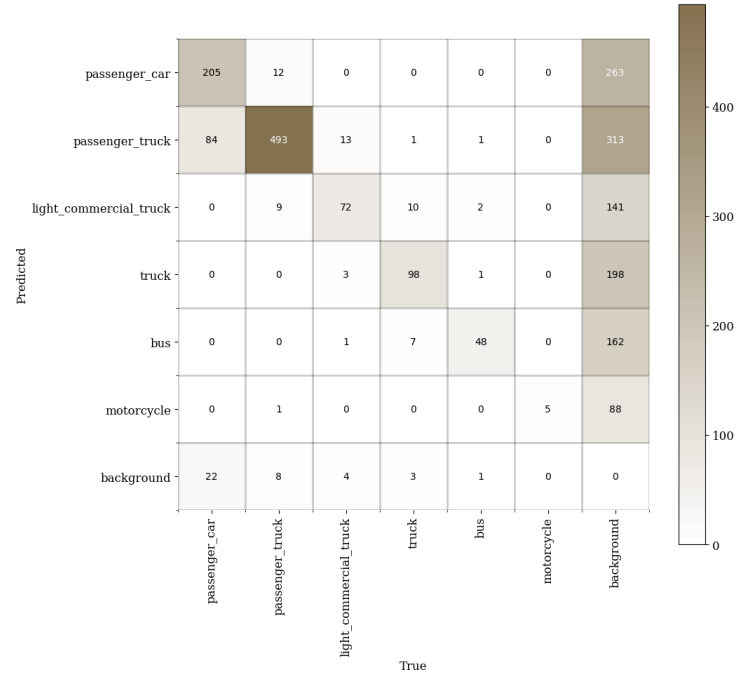
### 5.1.2 Data-Centric Refinements

#### Dataset Refinement

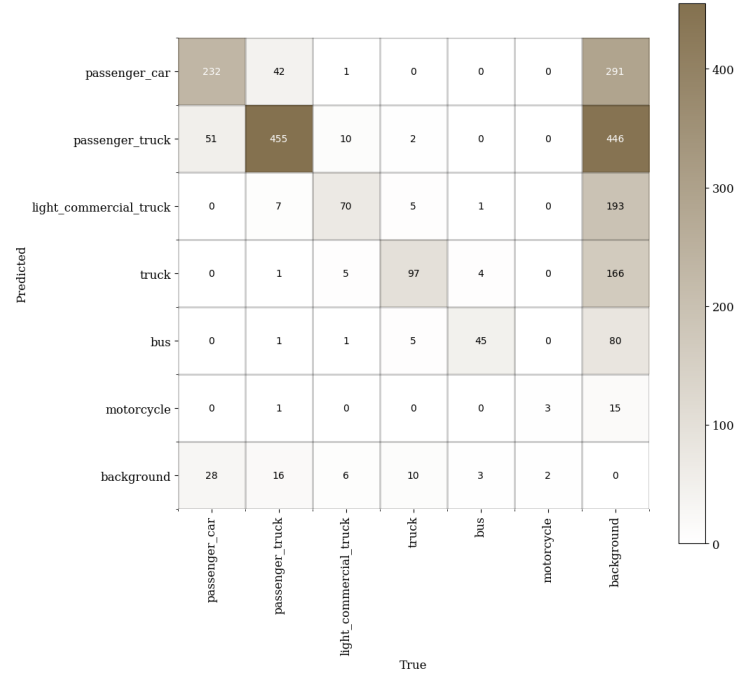
After establishing the baseline YOLO results, the next stage of experimentation focused on data-centric refinement guided by manual inspection of model errors. Rather than changing the detector architecture, this stage used the baseline confusion patterns to identify which types of examples required closer review. Particular attention was given to the dominant confusion between passenger car and passenger truck, as well as to vehicles that appeared visually ambiguous in the original imagery. Model predictions that did not match the validation set label were not used to relabel the dataset automatically, but instead served to flag candidate examples for manual review. The validation confusion matrices before dataset refinement for the baseline YOLOv11s and YOLOv26s models are shown in Figure 27. These predictions followed the same ignore-aware evaluation outlined in Section 4.3.

After this review, the dataset was refined by reassessing flagged examples individually. Vehicles whose labels could be determined more confidently from the imagery were reassigned to the appropriate class, while vehicles that remained too ambiguous were labeled as `unknown_vehicle` and treated as ignore regions. The post-refinement validation confusion matrices for the YOLOv11s and YOLOv26s models are shown in Figure 28. Relative to the pre-refinement results, these matrices show larger values along the main diagonal and smaller values in many off-diagonal cells, suggesting improved class consistency after dataset refinement. In particular, the number of cases in which passenger cars were predicted for background regions was reduced.

As shown in Table 10, the dataset refinement step led to small but consistent improvements in validation performance for both baseline models. YOLOv11s improved from a mAP50 of 0.804 to 0.814 and from a mAP50–95 of 0.615 to 0.622, while YOLOv26s improved from a mAP50 of 0.802 to 0.809 and from a mAP50–95 of 0.632 to 0.637. These gains were modest, but they support the conclusion that manually reassessing flagged examples and preserving genuinely ambiguous vehicles as `unknown_vehicle` improved dataset quality.

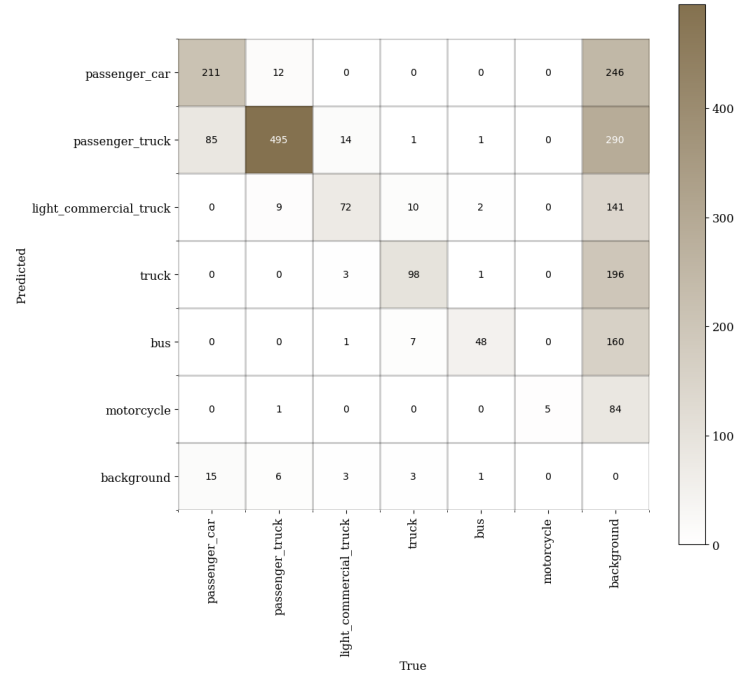


(a) Validation confusion matrix for the baseline YOLOv11s model before dataset refinement.

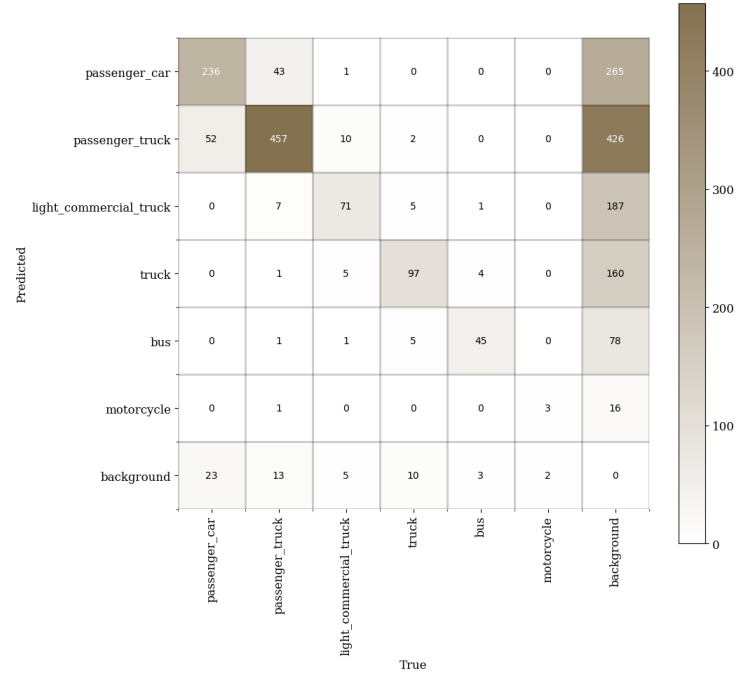


(b) Validation confusion matrix for the baseline YOLOv26s model before dataset refinement.

Figure 27: Ignore-aware validation confusion matrices for the baseline YOLOv11s and YOLOv26s models prior to dataset refinement.



(a) Validation confusion matrix for the baseline YOLOv11s model after dataset refinement.



(b) Validation confusion matrix for the baseline YOLOv26s model after dataset refinement.

Figure 28: Ignore-aware validation confusion matrices for the baseline YOLOv11s and YOLOv26s models after dataset refinement.

Table 10: Validation performance before and after dataset refinement for the baseline YOLOv11s and YOLOv26s models. The best mAP50 and mAP50–95 values for each model are displayed in bold.

| Base Model | Dataset Version | mAP50        | mAP50–95     |
|------------|-----------------|--------------|--------------|
| YOLOv11s   | Pre-refinement  | 0.804        | 0.615        |
|            | Post-refinement | <b>0.814</b> | <b>0.622</b> |
| YOLOv26s   | Pre-refinement  | 0.802        | 0.632        |
|            | Post-refinement | <b>0.809</b> | <b>0.637</b> |

Table 11 summarizes the validation-set class counts before and after dataset refinement. The number of labeled instances increased slightly in most supervised classes, including passenger car, passenger truck, truck, bus, and motorcycle, while light commercial truck decreased by one instance.

Table 11: Validation-set class counts before and after dataset refinement.

| Class                  | Pre-Refinement | Post-Refinement |
|------------------------|----------------|-----------------|
| passenger_car          | 304            | 311             |
| passenger_truck        | 521            | 524             |
| light_commercial_truck | 94             | 93              |
| truck                  | 119            | 120             |
| bus                    | 50             | 53              |
| motorcycle             | 4              | 6               |
| unknown_vehicle        | 833            | 928             |
| all                    | 1925           | 2035            |

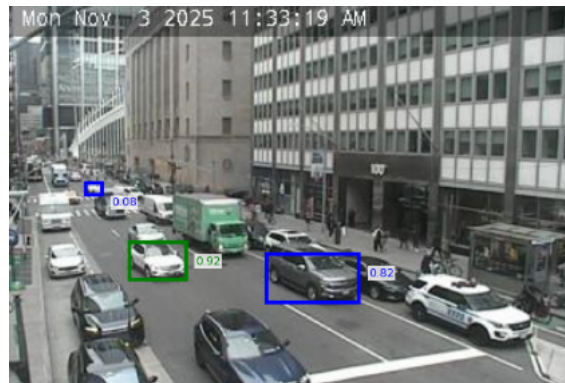
## Hard Negative Mining

The primary refinement strategy that followed was hard negative mining. As indicated by the post-refinement validation confusion matrices in Figure 28, the dominant remaining error for both baseline models was the confusion between passenger cars and passenger trucks. To target this directly, the training set was scanned using the baseline YOLOv11s model for YOLOv11s refinement and the baseline YOLOv26s model for YOLOv26s refinement to identify images in which a ground-truth passenger car was predicted as a passenger truck, or a ground-truth passenger truck was predicted as a passenger car. In this mining step, each ground-truth passenger car or passenger truck was matched to its highest-overlap prediction, and an image was flagged when that match exceeded an IoU threshold of 0.50 but was assigned to the opposite class. Figure 29 shows an example of a

hard negative identified by this procedure, in which vehicles were confused between the passenger car and passenger truck classes.



(a) Original training image before hard negative visualization.



(b) Same image with hard negative passenger car / passenger truck confusions identified by the mining procedure.

Figure 29: Example of a hard negative identified during passenger car / passenger truck mining with the YOLOv11s baseline. Subfigure (a) shows the original training image, while subfigure (b) shows the same image with class-swap detections highlighted. Blue boxes denote passenger trucks predicted as passenger cars, and green boxes denote passenger cars predicted as passenger trucks. The number next to the bounding boxes represents the prediction confidence score. These top images containing the most difficult boundary cases were duplicated back into the training set during retraining so that the detector would be exposed more frequently to the most informative class confusions.

A sweep was performed over the mining confidence threshold and the number of difficult images retained, and the strongest configuration used a confidence threshold of 0.05 with the top 100 difficult images added back into the training set. This targeted refinement produced the best-performing detector in the study. Compared with the baseline YOLOv11s experiment, training a new YOLOv11s model on the original training set augmented with the mined hard examples improved validation performance from mAP50 of 0.804 to 0.857 and from mAP50–95 of 0.615 to 0.661. This was the strongest overall result obtained among the YOLO experiments.

Additional data-centric experiments were also explored through selective oversampling of weaker classes, including `light_commercial_truck`, `truck`, and `motorcycle`. However, these broader class-specific duplication strategies did not improve upon the best hard-negative-mined configuration. A local grid search was also performed around the baseline hyperparameter settings to determine whether further gains could be obtained through optimization alone, but none of the tested configurations improved upon the best hard-negative-mined YOLOv11s model in either mAP50 or mAP50–95. Comparable

hard-negative experiments were also conducted using YOLOv26s, with additional variation in the mining confidence threshold and the number of selected images. Although these runs produced small improvements, none exceeded the best refined YOLOv11s result. Further refinement runs were also carried out by initializing training from previously trained baseline and refined YOLOv11s and YOLOv26s checkpoints rather than from the standard pretrained Ultralytics weights. In these cases, training generally plateaued at lower mAP50 and mAP50–95 values, even when hyperparameters were varied, indicating that continued fine-tuning from existing checkpoints was less effective than retraining a new model on the augmented dataset. Selected strong-performing YOLO results across the YOLOv11s and YOLOv26s backbones, including baseline, mined, oversampled, hyperparameter-search, and continued fine-tuning configurations, are summarized in Table 12.

Table 12: Summary of the YOLO refinement experiments conducted under the MOVES-based taxonomy. mAP50 and mAP50–95 are reported at the best validation checkpoint for each run. Extra images refers to the additional images added to the training set.

| Backbone | Extra<br>Images | Hyperparameters | mAP50 | mAP50–95 | Notes                                                                                                                                                                                                                                                                               |
|----------|-----------------|-----------------|-------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| YOLOv11s | 0               | Default YOLO    | 0.804 | 0.615    | Baseline YOLOv11s model.                                                                                                                                                                                                                                                            |
| YOLOv11s | 100             | Default YOLO    | 0.857 | 0.661    | Added 100 mined <code>passenger_car</code> ↔ <code>passenger_truck</code> hard negatives identified using the baseline YOLOv11s model.                                                                                                                                              |
| YOLOv11s | 400             | Default YOLO    | 0.776 | 0.597    | Added mined <code>passenger_car</code> ↔ <code>passenger_truck</code> hard negatives identified using the baseline YOLOv11s model. Additionally oversampled 300 images containing <code>light_commercial_truck</code> , <code>truck</code> , and <code>motorcycle</code> instances. |

Continued on next page



| Backbone | Extra<br>Images | Hyperparameters                                                                       | mAP50 | mAP50-95 | Notes                                                                                                                                                                                                                                                                                                       |
|----------|-----------------|---------------------------------------------------------------------------------------|-------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| YOLOv11s | 1000            | lr0 = 0.002;<br>lrf = 0.01;<br>close_mosaic = 10;<br>mixup = 0.0;<br>copy_paste = 0.0 | 0.831 | 0.648    | Fine-tuned from the best-performing YOLOv11s model after adding mined <code>passenger_car</code> ↔ <code>passenger_truck</code> hard negatives, 830 oversampled <code>light_commercial_truck</code> and <code>truck</code> examples, and 70 additional images containing <code>motorcycle</code> instances. |
| YOLOv11s | 100             | Default YOLO with horizontal flipping disabled                                        | 0.856 | 0.657    | Added 100 mined <code>passenger_car</code> ↔ <code>passenger_truck</code> hard negatives identified using the baseline YOLOv11s model. Best non-default local hyperparameter-search run.                                                                                                                    |
| YOLOv26s | 0               | Default YOLO                                                                          | 0.802 | 0.632    | Baseline YOLOv26s model.                                                                                                                                                                                                                                                                                    |
| YOLOv26s | 100             | Default YOLO                                                                          | 0.818 | 0.637    | Added 100 mined <code>passenger_car</code> ↔ <code>passenger_truck</code> hard negatives identified using the baseline YOLOv26s model.                                                                                                                                                                      |
| YOLOv26s | 100             | lr0 = 0.001;<br>cosine LR;<br>close_mosaic = 10;<br>mixup disabled;<br>freeze = 10    | 0.843 | 0.640    | Fine-tuned from the baseline YOLOv26s model after adding 100 mined <code>passenger_car</code> ↔ <code>passenger_truck</code> hard negatives.                                                                                                                                                                |

Overall, the data-centric refinement experiments showed that the largest performance gains were obtained from adding mined hard examples targeting the `passenger_car` and `passenger_truck` boundary, whereas broader class-specific oversampling and additional hyperparameter changes did not yield further improvement.

### 5.1.3 Final YOLO Model Selection and Test-Set Evaluation

Based on the baseline comparisons and subsequent data-centric refinement experiments, the final detector selected for this study was YOLOv11s, saved as checkpoint `vehicle_6c_v11s11`. For readability, this checkpoint is referred to throughout the remainder of the thesis as the final YOLOv11s model. Among all YOLO experiments, this configuration achieved the strongest validation performance, with a mAP50 of 0.857 and a mAP50–95 of 0.661.

Although additional refinement runs were explored after this stage, none improved upon the validation performance of the final YOLOv11s model. This model was therefore selected for all remaining evaluations and was assessed once on the held-out test set, which had been reserved exclusively for final evaluation. The test set was evaluated using the same ignore-aware procedure used for validation, as outlined in Section 4.3. The resulting test set performance is summarized by class in Table 13.

Table 13: Held-out test set performance of the final YOLOv11s model, reported by class.

| Class                  | Images | Instances | AP50  | AP50–95 |
|------------------------|--------|-----------|-------|---------|
| passenger_car          | 114    | 363       | 0.779 | 0.620   |
| passenger_truck        | 117    | 643       | 0.868 | 0.705   |
| light_commercial_truck | 62     | 110       | 0.867 | 0.703   |
| truck                  | 42     | 65        | 0.926 | 0.701   |
| bus                    | 44     | 67        | 0.895 | 0.703   |
| motorcycle             | 5      | 5         | 0.298 | 0.209   |
| all                    | 120    | 1253      | 0.772 | 0.607   |

With the final YOLOv11s model selected and evaluated on the held-out test set, the next section compares these YOLO results with those obtained from alternative detection architectures.

## 5.2 Comparison with Alternative Detection Architectures

### 5.2.1 Faster R-CNN

Like the YOLO experiments, Faster R-CNN experiments were conducted under the MOVES-based taxonomy. Faster R-CNN is a two-stage object detector that first generates candidate object regions using a Region Proposal Network and then classifies and refines

those regions in a second stage [67]. For implementation, Torchvision Faster R-CNN models with ResNet-50 feature pyramid backbones were used, including both the standard `fasterrcnn_resnet50_fpn` model and the newer `fasterrcnn_resnet50_fpn_v2` variant provided in Torchvision. Feature pyramid backbones are commonly used to improve multi-scale object detection [68], and the specific torchvision implementation is documented by the torchvision maintainers [69].

For the Faster R-CNN experiments, the detector head was adapted to the MOVES-based taxonomy by replacing the default box predictor with a new predictor for six foreground classes plus background. The training pipeline used the same train/validation split as the other detectors. To better match the small vehicles that appear in the traffic-camera images, the anchor generator used box sizes of 8, 16, 32, 64, and 128 pixels across the feature pyramid. As in the YOLO experiments, images were resized to  $768 \times 1280$ . Training used AdamW with a learning rate of  $2 \times 10^{-4}$ , weight decay of  $10^{-4}$ , and a cosine annealing learning-rate schedule over 50 epochs. Light augmentation was applied during training, consisting of random horizontal flipping and modest color jitter in the best runs. Batch size was set to 4 for training and 2 for validation. As in the YOLO and transformer experiments, `unknown_vehicle` instances were masked during training and treated as ignore regions during evaluation, following the same ignore-aware procedure outlined in Section 4.3.

A light targeted grid search was conducted for the Faster R-CNN experiments to evaluate a small number of influential hyperparameters without the cost of a full exhaustive sweep. The search varied four factors: backbone variant (`fasterrcnn_resnet50_fpn_v2` versus `fasterrcnn_resnet50_fpn`), learning rate ( $1 \times 10^{-4}$  and  $2 \times 10^{-4}$ ), augmentation strategy (random horizontal flipping only versus horizontal flipping with modest color jitter), and the number of trainable backbone layers (3 versus 5). Weight decay was held fixed at  $1 \times 10^{-4}$ , and each run was trained for 50 epochs under the same general pipeline. Table 14 summarizes the strongest Faster R-CNN results produced in this study.

Table 14: Validation performance of the Faster R-CNN experiments under the MOVES-based taxonomy.

| Backbone         | Learning Rate | Augmentation        | Trainable<br>Backbone<br>Layers | mAP50 | mAP50–95 |
|------------------|---------------|---------------------|---------------------------------|-------|----------|
| ResNet-50-FPN    | 1e-4          | Flip + Color Jitter | 5                               | 0.658 | 0.440    |
| ResNet-50-FPN-v2 | 2e-4          | Flip + Color Jitter | 5                               | 0.706 | 0.509    |

Among the tested configurations, the best overall Faster R-CNN result was obtained with the `fasterrcnn_resnet50_fpn_v2` backbone using a learning rate of  $2 \times 10^{-4}$ , flip-plus-color-jitter augmentation, and 5 trainable backbone layers. On the validation set, this model achieved a mAP50 of 0.706 and a mAP50–95 of 0.509. Other strong `fasterrcnn_resnet50_fpn_v2` runs achieved mAP50 values of 0.693 and 0.686, with corresponding mAP50–95 values of 0.500 and 0.497, indicating that the v2 backbone was consistently the strongest Faster R-CNN variant across the tested settings. By comparison, the best standard `fasterrcnn_resnet50_fpn` configuration achieved a mAP50 of 0.658 and a mAP50–95 of 0.440.

Although the `fasterrcnn_resnet50_fpn_v2` experiments produced the strongest Faster R-CNN results in this study, their validation performance remained below that of the YOLO baselines reported in Table 9. The best Faster R-CNN configuration achieved a mAP50 of 0.706 and a mAP50–95 of 0.509, whereas the strongest baseline YOLO results were already higher before additional refinement. Since the Faster R-CNN baselines did not approach the performance of the leading YOLO configurations, further iteration on the Faster R-CNN models was not pursued.

### 5.2.2 Detection Transformer (DETR)

Three transformer-based detectors were evaluated under the MOVES-based taxonomy: SenseTime Deformable DETR, Peking University RT-DETR, and Peking University RT-DETRv2. These models were selected to provide a comparison across recent DETR-style detector designs within a common implementation framework. Deformable DETR served as an established multi-scale transformer baseline, while RT-DETR and RT-DETRv2 represented more efficiency-oriented DETR variants intended to improve the speed-accuracy tradeoff [70]–[73].

For these experiments, the detector backbone was kept frozen for the first 10 epochs and then unfrozen for the remainder of training, with the full model trained for 50 epochs using AdamW and a weight decay of  $10^{-4}$ . A cosine learning-rate schedule was used, and learning rates were selected through iterative tuning rather than exhaustive search, since a single training run required several hours. Parameter settings were chosen by monitoring whether validation performance remained stable and continued improving over training, while avoiding configurations that caused the detector to stall or degrade early. The best learning rates for Deformable DETR and RT-DETR are summarized in Table 15.

Training augmentation was ultimately kept light, consisting primarily of horizontal flipping and modest brightness/contrast perturbations. This decision was motivated by earlier experiments using a heavier Albumentations pipeline, which suggested that transformer-based detectors were sensitive to stronger image transformations in this low-resolution traffic-camera setting. In this section, heavier Albumentations pipeline refers to the augmentation setup used in the first Deformable DETR experiment, and lighter Albumentations pipeline refers to the simplified setup used in the subsequent experiments. The exact transform definitions are listed in Appendix A. As in the YOLO and R-CNN experiments, `unknown_vehicle` instances were masked during training, and treated as ignore regions during evaluation.

Table 15: Validation performance of the transformer-based detector experiments under the final 6-class formulation. All runs were trained for 50 epochs with 10 warmup epochs.

| Backbone        | Head LR<br>(Frozen) | Backbone LR<br>(Unfrozen) | Head LR<br>(Unfrozen) | Augmentations | mAP50 | mAP50-95 |
|-----------------|---------------------|---------------------------|-----------------------|---------------|-------|----------|
| Deformable DETR | $1 \times 10^{-4}$  | $1 \times 10^{-5}$        | $7 \times 10^{-5}$    | Heavy         | 0.428 | 0.318    |
| Deformable DETR | $1 \times 10^{-4}$  | $1 \times 10^{-5}$        | $7 \times 10^{-5}$    | Light         | 0.564 | 0.421    |
| RT-DETR         | $1 \times 10^{-5}$  | $5 \times 10^{-6}$        | $7 \times 10^{-6}$    | Light         | 0.596 | 0.437    |
| RT-DETRv2       | $1 \times 10^{-5}$  | $5 \times 10^{-6}$        | $7 \times 10^{-6}$    | Light         | 0.688 | 0.516    |

The transformer experiments showed a clear progression across the tested DETR variants. For SenseTime Deformable DETR, an initial 50-epoch run achieved a mAP50 of 0.428 and a mAP50-95 of 0.318, while a second run with less aggressive image transforms improved performance to 0.564 mAP50 and 0.421 mAP50-95. Additional hyperparameter and augmentation adjustments were also tested, but none exceeded this result. Under the same 50-epoch training schedule, RT-DETR achieved a mAP50 of 0.596 and a mAP50-95 of 0.437, and RT-DETRv2 produced the strongest transformer-based result in this study, reaching a mAP50 of 0.688 and a mAP50-95 of 0.516.

Although RT-DETRv2 was the strongest transformer-based model evaluated, its performance still remained below the baseline YOLO results reported in Table 9. Accordingly, the DETR models were not pursued further as candidates for the final detector.

### 5.3 Model Robustness and Cross-Camera Generalization

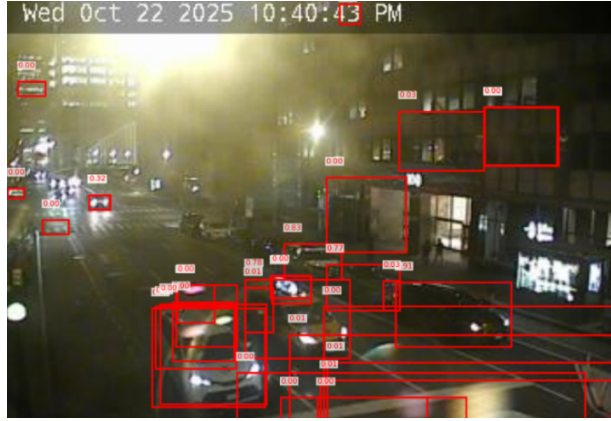
After selecting the final YOLOv11s detector, additional experiments were conducted to evaluate its robustness and generalization beyond the original validation split. These experiments focused on two questions. First, the sensitivity of the detector to confidence-threshold and non-maximum-suppression settings was examined to determine how stable performance remained under different inference configurations. Second, cross-camera generalization was assessed using a manually annotated benchmark of 75 images collected from three previously unseen inference cameras. These images were sampled between March 12 and March 20, 2026, with 25 images per camera selected at varying times of day in order to capture a range of traffic and viewing conditions.

#### 5.3.1 Confidence Threshold and NMS Sensitivity Analysis

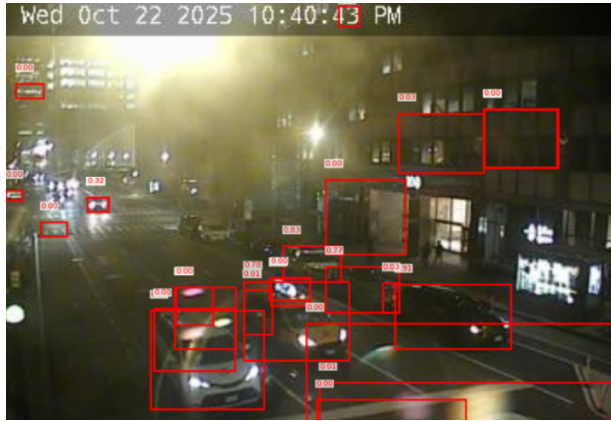
To evaluate how dependent the final detector was on inference-time settings, a confidence-threshold and non-maximum suppression (NMS) sensitivity analysis was performed using the final YOLOv11s model. In this context, the confidence threshold determines the minimum prediction confidence required for a detection to be kept, while the NMS IoU threshold controls how aggressively overlapping detections are suppressed during post-processing [74]. Lower confidence thresholds retain more candidate detections, typically increasing recall at the expense of additional false positives, whereas higher thresholds are more selective. Similarly, lower NMS IoU thresholds suppress overlapping boxes more aggressively, while higher values allow more overlapping detections to remain.

This tradeoff is illustrated qualitatively in Figure 30. Each panel shows the same traffic scene under a different inference configuration, with red boxes indicating detections and adjacent numbers indicating confidence scores. In Figure 30a, a very low confidence threshold (`conf` = 0.001) and a relatively permissive NMS IoU threshold (`det_iou` = 0.70) retain many detections, including several overlapping boxes around the same vehicles. In Figure 30b, the confidence threshold remains at 0.001 but the NMS IoU threshold is reduced to 0.40, producing fewer overlapping detections. In Figure 30c, the NMS IoU threshold remains at 0.40 while the confidence threshold is increased to 0.10, removing many weaker detections and leaving a sparser set of predictions. Together, these examples show how stricter inference settings can reduce duplicate detections and visual clutter, but may also suppress vehicles that would otherwise be counted.

To examine the stability of the detector across a reasonable operating range, the confidence



(a) Same scene evaluated with `conf = 0.001` and `det_iou = 0.70`.



(b) Same scene evaluated with `conf = 0.001` and `det_iou = 0.40`.



(c) Same scene evaluated with `conf = 0.10` and `det_iou = 0.40`.

Figure 30: Qualitative example of confidence-threshold and NMS sensitivity for the final YOLOv11s model. Each panel shows the same traffic scene under a different inference configuration, with red boxes indicating retained detections and adjacent numbers indicating confidence scores. Lowering the NMS IoU threshold reduces overlapping detections, while increasing the confidence threshold further suppresses weaker predictions.

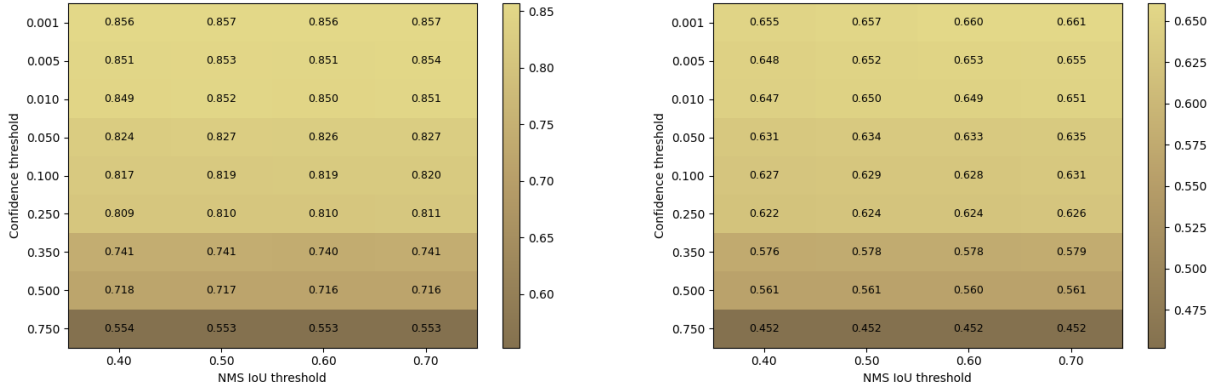
threshold was swept over:

$$\{0.001, 0.005, 0.01, 0.05, 0.10, 0.25, 0.35, 0.50, 0.75\}$$

and the NMS IoU threshold was swept over

$$\{0.40, 0.50, 0.60, 0.70\}.$$

For each combination, predictions were evaluated on the validation set using the same ignore-aware evaluation procedure described in Section 4.3, in which detections overlapping `unknown_vehicle` regions above the specified ignore IoU threshold were removed before scoring. This produced a grid of 36 inference configurations, allowing the sensitivity of both mAP50 and mAP50–95 to be assessed for tuning without using the held-out test set. Figure 31 presents heatmaps of validation mAP50 and mAP50–95 across the evaluated confidence-threshold and NMS IoU grid for the final YOLOv11s model.



(a) Validation mAP50 across confidence-threshold and NMS IoU settings for the final YOLOv11s model.

(b) Validation mAP50–95 across confidence-threshold and NMS IoU settings for the final YOLOv11s model.

Figure 31: Heatmaps showing the sensitivity of validation performance to inference-time confidence and NMS IoU settings for the final YOLOv11s model. Subfigure (a) shows mAP50 and subfigure (b) shows mAP50–95 over the evaluated confidence-threshold and NMS IoU grid.

As shown in Figure 31, both mAP50 and mAP50–95 were relatively stable across the lower confidence-threshold range, with the strongest values occurring near `conf` = 0.001 and higher `det_iou` settings. Performance then declined gradually as the confidence threshold increased to 0.25, followed by a more noticeable drop beginning at `conf` = 0.35 and a sharper decrease at `conf` = 0.75. By contrast, variation across the NMS IoU threshold was



comparatively small throughout the sweep, with changes in `det_iou` producing only modest differences relative to the effect of the confidence threshold. These results indicate that detector performance was much more sensitive to aggressive confidence filtering than to the specific NMS IoU setting within the tested range.

### 5.3.2 Cross-Camera Generalization

To assess how well the final YOLO detector can predict vehicle classes beyond the original train-validation-test split, a separate manually annotated benchmark was constructed using images from three traffic cameras unseen by the model during training. These cameras were located at Atlantic Avenue at Boerum Place, Broadway at Houston, and Pelham Parkway Westbound at Boston Road. These locations were selected not only to provide unseen camera views for detector evaluation, but also because they were later used in the traffic-count comparison in Section 5.4.

The benchmark contained 75 images in total, with 25 images sampled from each camera. These images were drawn from March 12 to March 20, 2026, and were selected across different times of day in order to capture a broader range of traffic density, lighting, and scene conditions. The resulting benchmark was intended to better approximate deployment conditions than the original validation set, since none of these camera views were included during training. The full class distribution of the benchmark is summarized in Table 16.

Table 16: Class distribution of the 75-image cross-camera benchmark.

| Class                  | Instances |
|------------------------|-----------|
| passenger_car          | 107       |
| passenger_truck        | 209       |
| light_commercial_truck | 25        |
| truck                  | 13        |
| bus                    | 8         |
| motorcycle             | 2         |
| unknown_vehicle        | 474       |
| all                    | 838       |

The cross-camera benchmark was evaluated using the same ignore-aware procedure as described in Section 4.3. Table 17 summarizes the per-class results on this 75-image benchmark. Overall, the final YOLOv11s model achieved a mAP50 of 0.658 and a mAP50–95 of 0.488 across 364 annotated vehicle instances.

Table 17: Per-class detection performance of the final YOLOv11s model on the 75-image cross-camera benchmark.

| Class                  | AP50  | AP50-95 |
|------------------------|-------|---------|
| passenger_car          | 0.626 | 0.487   |
| passenger_truck        | 0.761 | 0.592   |
| light_commercial_truck | 0.685 | 0.488   |
| truck                  | 0.735 | 0.514   |
| bus                    | 0.635 | 0.490   |
| motorcycle             | 0.505 | 0.353   |
| all                    | 0.658 | 0.488   |

## 5.4 Comparison with ATR Traffic Counts

To evaluate whether the detector-derived counts reflected real traffic patterns, hourly vehicle counts produced by the final YOLOv11s model were compared with New York City automated traffic recorder (ATR) counts [10] at three unseen camera locations: Atlantic Avenue at Boerum Place, Broadway at Houston, and Pelham Parkway Westbound at Boston Road. These were the same three locations used in the cross-camera benchmark in Section 5.3.2, allowing the detector’s performance on unseen views to be examined further through a traffic-count comparison.

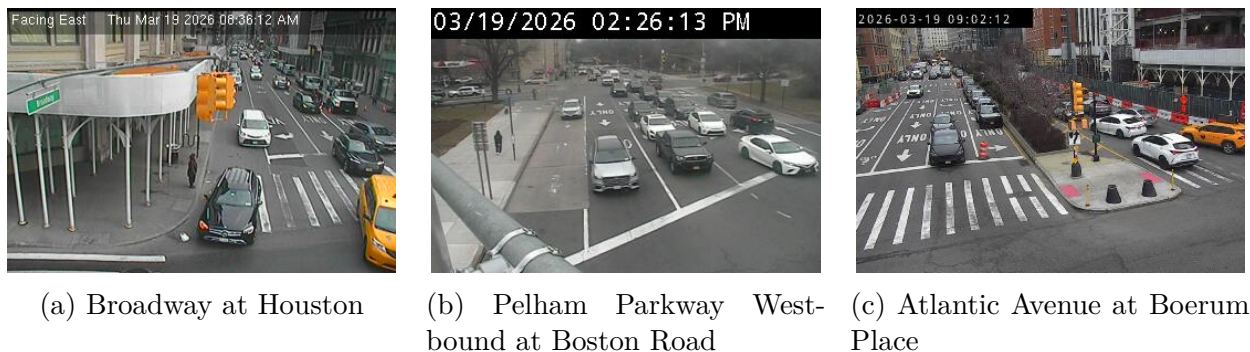


Figure 32: New camera imagery used for inference.

The three camera locations were selected because they were positioned on urban streets rather than major highways, which were not the primary focus of this study. Because March 2026 ATR data were not available at the time the camera imagery was collected, the comparison instead used March 2025 ATR records from locations where an active traffic camera was available at the same street intersection. March 2026 camera imagery was then matched to March 2025 ATR dates with broadly similar weather conditions [75].

Images were collected from March 13, 2026 at 12:01 AM through March 20, 2026 at 12:00 PM. An image was sampled from each camera every two minutes, consistent with the sampling rate used during dataset creation, to reduce the risk of the same scene being counted more than once, since the camera feeds update slowly and explicit image-to-image vehicle matching was not implemented. Table 18 summarizes the camera locations, matched comparison dates, and total number of collected images.

Table 18: Camera locations, comparison dates, and total number of collected images used in the ATR traffic-count analysis.

| Camera Intersection         | Comparison Dates         |              |
|-----------------------------|--------------------------|--------------|
|                             | (2026 images / 2025 ATR) | Total Images |
| Broadway @ Houston          | 3/14 – 3/17              | 2,774        |
| Pelham Pkwy W/B @ Boston Rd | 3/13 – 3/17              | 3,488        |
| Atlantic Ave @ Boerum Place | 3/18 – 3/19              | 1,440        |

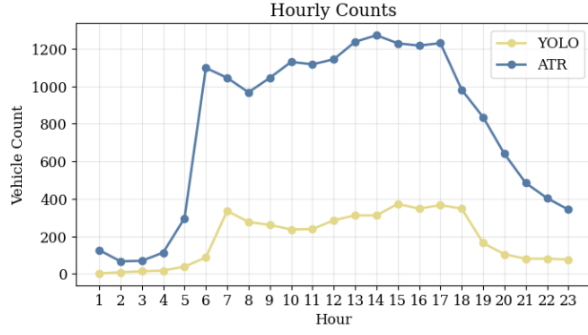
After the images were collected, inference was run over all images using the final YOLOv11s model. Based on the validation heatmaps in Figure 31, performance remained relatively stable at lower confidence thresholds but declined more noticeably beyond 0.25, so a confidence threshold of 0.25 and an NMS IoU threshold of 0.70 were used for inference. The resulting detections were aggregated into hourly vehicle counts and matched to ATR hourly traffic volumes by camera-specific road segment and hour of day. Because the ATR data were reported in 15-minute intervals, those records were first summed to hourly totals before comparison. Table 19 provides a per-day summary at each camera location, comparing the YOLO-derived counts with the ATR counts.

Table 19: Per-day comparison of YOLO-derived and ATR vehicle counts across the selected camera locations, including hourly-profile Pearson correlation, mean absolute error, and mean absolute percentage error.

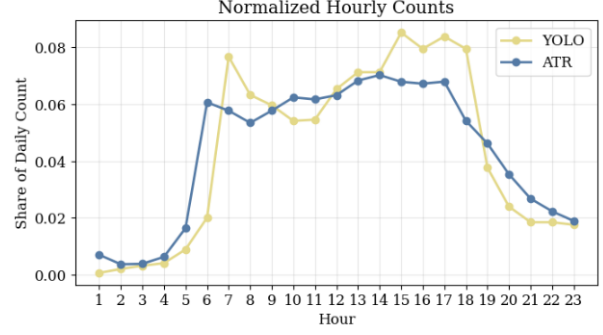
| Camera Intersection         | Date      | YOLO Count (2026) | ATR Count (2025) | Pearson Correlation | MAE    | MAPE (%) |
|-----------------------------|-----------|-------------------|------------------|---------------------|--------|----------|
| Broadway @ Houston          | 3/14/2026 | 4,752             | 16,252           | 0.608               | 479.17 | 68.29    |
| Broadway @ Houston          | 3/15/2026 | 4,709             | 15,626           | 0.597               | 454.88 | 69.08    |
| Broadway @ Houston          | 3/16/2026 | 3,431             | 12,382           | 0.234               | 393.17 | 67.22    |
| Broadway @ Houston          | 3/17/2026 | 3,910             | 13,067           | 0.753               | 398.13 | 68.31    |
| Pelham Pkwy W/B @ Boston Rd | 3/13/2026 | 5,793             | 20,043           | 0.892               | 593.75 | 70.66    |
| Pelham Pkwy W/B @ Boston Rd | 3/14/2026 | 4,947             | 20,423           | 0.866               | 644.83 | 70.78    |
| Pelham Pkwy W/B @ Boston Rd | 3/15/2026 | 4,181             | 17,742           | 0.897               | 565.04 | 76.15    |
| Pelham Pkwy W/B @ Boston Rd | 3/16/2026 | 4,107             | 13,791           | 0.780               | 421.04 | 71.35    |
| Pelham Pkwy W/B @ Boston Rd | 3/17/2026 | 4,376             | 18,096           | 0.906               | 596.52 | 78.47    |
| Atlantic Ave @ Boerum Place | 3/18/2026 | 7,559             | 8,725            | 0.844               | 82.67  | 22.96    |
| Atlantic Ave @ Boerum Place | 3/19/2026 | 7,060             | 9,188            | 0.803               | 104.17 | 27.28    |

As shown in Table 19, agreement with ATR counts varied substantially across locations. The closest match between counts was observed at Atlantic Avenue at Boerum Place, where detector totals of 7,060 to 7,559 were compared with ATR totals of 8,725 to 9,188, with mean absolute percentage error values of 22.96% and 27.28%. Broadway at Houston showed larger discrepancies, with detector totals of 3,431 to 4,752 versus ATR totals of 12,382 to 16,252 and mean absolute percentage error values ranging from 67.22% to 69.08%. Pelham Parkway Westbound at Boston Road exhibited the largest undercount overall, with detector totals of 4,107 to 5,793 compared with ATR totals of 13,791 to 20,423 and mean absolute percentage error values ranging from 70.66% to 78.47%.

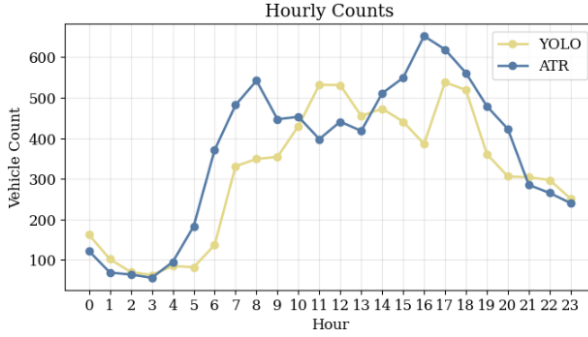
In addition to total-count error, Pearson correlation was computed for each camera-day pair using normalized hourly YOLO and ATR counts, where each hour was normalized relative to the source’s total daily vehicle count. This was done to compare how closely the two sources followed the same hourly traffic pattern, independent of differences in overall count size. Under this measure, Atlantic Avenue at Boerum Place showed consistently strong hourly agreement, with correlation values of 0.844 and 0.803. Broadway at Houston showed weaker and less consistent agreement, with values ranging from 0.234 to 0.753. Pelham Parkway Westbound at Boston Road generally showed strong hourly-pattern agreement despite larger differences in total volume, with correlations ranging from 0.780 to 0.906. See Figure 33 for hourly count comparisons for selected camera-day pairs, showing both the raw hourly counts and the normalized hourly counts for the YOLO and ATR detections. Overall, these results showed that the final YOLOv11s model reproduced hourly traffic patterns more consistently at some unseen camera locations than others, while agreement with ATR count magnitude remained strongly location-dependent.



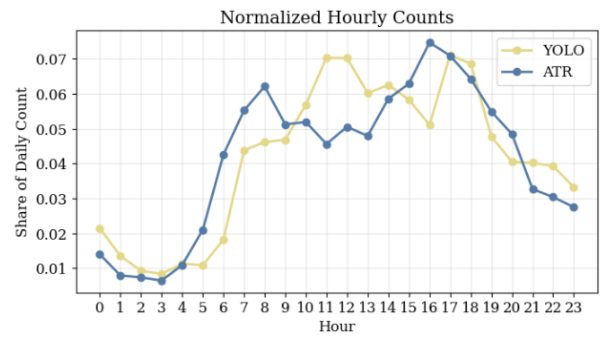
(a) Pelham Parkway Westbound at Boston Road (3/17). Raw hourly counts.



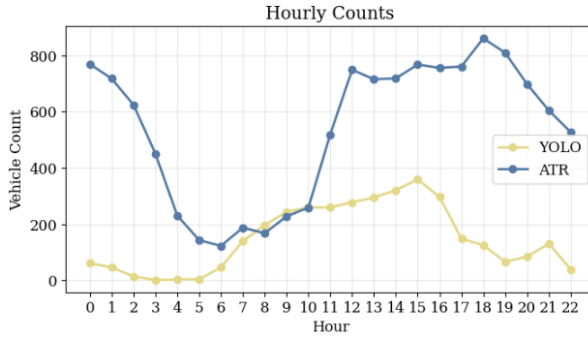
(b) Pelham Parkway Westbound at Boston Road (3/17). Normalized hourly counts.



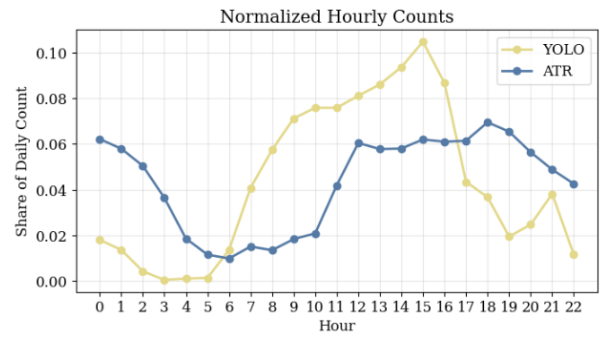
(c) Atlantic Avenue at Boerum Place (3/18). Raw hourly counts.



(d) Atlantic Avenue at Boerum Place (3/18). Normalized hourly counts.



(e) Broadway at Houston (3/16). Raw hourly counts.



(f) Broadway at Houston (3/16). Normalized hourly counts.

Figure 33: Hourly detector-derived and ATR traffic-count comparisons for the selected camera-day pairs. The plots compare YOLO-derived counts from March 2026 against matched ATR counts from March 2025. Subfigures (a) and (b) show Pelham Parkway Westbound at Boston Road, with a YOLO-derived daily total of 4,376 and an ATR total of 18,096, and a Pearson correlation of 0.906. Subfigures (c) and (d) show Atlantic Avenue at Boerum Place, with a YOLO-derived daily total of 7,559 and an ATR total of 8,725, and a Pearson correlation of 0.844. Subfigures (e) and (f) show Broadway at Houston, with a YOLO-derived daily total of 3,431 and an ATR total of 12,382, and a Pearson correlation of 0.234. The left column shows raw hourly counts, while the right column shows normalized hourly profiles, with each hour expressed as a fraction of that source’s daily total.

## 6 Discussion

The experimental framework established in this research serves to validate the feasibility of utilizing New York City’s existing traffic surveillance infrastructure for high-resolution vehicle classification and emissions estimation. The results demonstrate that an optimized vehicle detection pipeline, specifically one centered on the YOLOv11s architecture, can effectively address many of the visual challenges inherent in low-resolution,  $352 \times 240$  traffic imagery. This discussion synthesizes the technical performance of the evaluated models, the importance of the MOVES-based taxonomy, and the broader implications of these findings for scalable environmental health monitoring in dense urban environments.

### 6.1 Evaluation of Detection Architectures for NYC Imagery

The selection of a primary detection model for this study was driven by a rigorous comparison across three distinct paradigms: one-stage convolutional neural networks, two-stage networks, and transformer-based architectures. Here, one-stage detectors refer to models such as YOLO that predict object locations and classes directly from the image in a single pass, whereas two-stage models first propose candidate object regions and then classify them. As shown in Table 20, the YOLO models performed consistently better than Faster R-CNN and the DETR models in terms of mAP50 and mAP50–95 on the validation set.

Table 20: Baseline validation performance of the principal detector architectures evaluated in this study.

| Model Architecture | mAP50        | mAP50–95     | Source   |
|--------------------|--------------|--------------|----------|
| YOLOv11s           | <b>0.804</b> | 0.615        | Table 9  |
| YOLOv26s           | 0.802        | <b>0.632</b> | Table 9  |
| YOLOv8s            | 0.750        | 0.606        | Table 9  |
| Faster R-CNN       | 0.706        | 0.509        | Table 14 |
| RT-DETRv2          | 0.688        | 0.516        | Table 15 |
| Deformable DETR    | 0.564        | 0.421        | Table 15 |

The performance differences across architectures suggest that, for low-quality  $352 \times 240$  traffic-camera images, an efficient detector design such as YOLO may be more effective than a larger, more complex architecture. Traditional two-stage detectors like Faster R-CNN try to locate candidate objects first and classify them second. However, across the 1,200-image dataset, this two-stage approach struggled to reliably detect the vehicles. Even

when the Faster R-CNN model was tuned (the `fasterrcnn_resnet50_fpn_v2` version) with 8 to 128-pixel anchors, it still could not beat the baseline performance scores (mAP50) of the YOLO models.

Furthermore, the transformer-based models (the DETR family) demonstrated a clear sensitivity to data volume and augmentation strategy. As noted in Section 5.2.2, Deformable DETR’s performance improved significantly (from 0.428 to 0.564 mAP50) when the image augmentation pipeline was lightened. This suggests that these types of models struggle to learn from a small dataset if the training images are too heavily modified or distorted. While newer variants like RT-DETRv2 showed promise, reaching a mAP50 of 0.688, they still remained less accurate than the YOLOv11s model. Ultimately, YOLO’s architecture proved more effective at extracting the necessary visual features from these low-resolution traffic scenes [61], [76].

The performance advantage of YOLOv11s observed in this study likely stems from its ability to efficiently process objects at multiple scales. As outlined in Section 4.1.6, its architecture combines visual features from different resolutions, which appears to help the model preserve semantic information for vehicles ranging from large buses in the foreground to small sedans in the distance. This multi-scale capability is particularly important for New York City’s traffic-camera imagery, where perspective, camera placement, and intersection geometry cause vehicles near the camera to occupy a large portion of the frame while vehicles farther down the street may appear as only a few pixels.

## 6.2 Rationale for the MOVES-Based Taxonomy

The performance of the final detector was contingent not only on the architecture, but on how the vehicle classes were defined. The MOVES-based taxonomy was created to balance what is actually visible in low-resolution traffic cameras with the data required by the MOVES emissions software. Instead of forcing the model to distinguish between vehicle body styles that look nearly identical at  $352 \times 240$  pixels, this approach grouped vehicles into broader categories based on their emissions profiles.

This strategy was applied from the beginning of the data annotation process. Visually similar body styles like coupes and hatchbacks were labeled as `sedan`, while crossovers were labeled as `suv`, relying on dependable visual cues like cabin height and rear profile sloping. In the final taxonomy, these labels were mapped directly to MOVES categories: sedans became `passenger_car`, while SUVs, pickup trucks, and minivans were grouped into

`passenger_truck`. This specific separation is critical for downstream environmental analysis, because EPA’s MOVES framework treats passenger cars, passenger trucks, and light commercial trucks as distinct on-road source types, and computes criteria-pollutant emissions such as  $NO_x$  and  $PM_{2.5}$  through source-type-specific emission-rate tables [14], [16].

The dataset statistics in Table 4 highlight two important properties of the annotated data. First, passenger cars and passenger trucks make up over 77% of all identified-vehicle annotations, making the boundary between these two categories especially important for model performance. Second, `unknown_vehicle` was the single largest category in the dataset, with 9,625 instances, reflecting the large number of vehicles that were visible but could not be assigned confidently to a supervised class because of distance, occlusion, blur, or limited viewing angle.

Using `unknown_vehicle` as an ignore region rather than a normal training class helped avoid forcing the model to learn from ambiguous examples. In supervised vision tasks, noisy or uncertain labels can hurt generalization, and object-detection benchmarks commonly exclude regions that cannot be annotated reliably from scoring rather than treating them as standard objects [77]. During training, these ambiguous regions were masked with a neutral fill value so the detector would not learn from pixels that human annotators could not label confidently. As a result, the reported mAP50 and mAP50–95 better reflected performance on clearly identifiable vehicles instead of penalizing the model for inherently uncertain regions.

### 6.3 Performance Gains from Targeted Data Refinement

The improvement from the baseline YOLOv11s model (mAP50 of 0.804) to the final selected version (mAP50 of 0.857) is consistent with recent work emphasizing that model performance can often be improved substantially through better data curation and refinement, even without changing the underlying architecture [78], [79]. The experiments in Section 5.1.2 suggest that the main limitation was not model capacity alone, but a concentrated weakness between predicting `passenger_car` and `passenger_truck`.

Manual inspection of the confusion matrices in Figure 27 and Figure 28 revealed that while the model had learned to localize vehicles with high precision, it frequently swapped labels between sedans and SUVs. The hard negative mining strategy in Section 5.1.2 addressed this by identifying 100 images where these swaps occurred and duplicating them back into



the training set. This targeted intervention proved significantly more effective than broad hyperparameter variation or general oversampling of rare classes like motorcycles.

Adding 400 extra images to the training set led to lower performance (mAP50 of 0.776) than the targeted 100-image hard negative configuration (mAP50 of 0.857). This shows that simply increasing dataset size is not enough. New data is most useful when it addresses the model’s existing failures. Here, the smaller hard negative set performed better because it directly reinforced the difficult boundary between similar vehicle classes. For urban monitoring applications, this suggests that iterative human-in-the-loop data refinement is more effective than untargeted dataset expansion. By reviewing the model’s own errors, the training set can be improved precisely where the detector is weakest.

## 6.4 Confidence Threshold Selection for Practical Inference

A key result from the threshold sensitivity analysis in Section 5.3.1 is that the validation metrics were driven much more by the confidence threshold than by the NMS IoU setting. Across all 36 settings in the sweep, the best performance occurred at the lowest confidence threshold tested, `conf` = 0.001 and `det_iou` = 0.7, where mAP50 reached 0.857 and mAP50–95 reached 0.661. As the confidence threshold increased, all three metrics fell steadily. At `conf` = 0.25, mAP50 was still about 0.809–0.811 and mAP50–95 was about 0.622–0.626, which is only a moderate drop from the theoretical peak. In contrast, changing `det_iou` from 0.4 to 0.7 had only a very small effect: the mean mAP50 across all thresholds changed from 0.780 to 0.781, while the mean mAP50–95 changed from 0.602 to 0.606. This shows that confidence filtering, not NMS tuning, was the dominant calibration variable.

This helps explain the large number of background false positives seen in the initial validation confusion matrices like Figure 27. Standard COCO-style AP evaluation rewards recall across the full precision-recall curve, so it naturally favors extremely low confidence thresholds such as `conf` = 0.001 [74]. In practice, however, this setting encourages the detector to keep many weak predictions, including boxes on ambiguous background textures, which creates visual clutter and ghost vehicles. The heatmap results show that raising the threshold to `conf` = 0.25 sacrifices only about 0.045 mAP50 and about 0.035 mAP50–95 relative to the best setting, while greatly improving the practical usability of the detections. For downstream tasks such as ATR volume comparison, this makes confidence thresholding an essential deployment step, as maximizing AP at very low confidence is not

automatically the setting that gives the cleanest and most reliable vehicle counts.

## 6.5 Cross-Camera Generalization and Domain Sensitivity

A key question for practical deployment is how well the final detector generalizes beyond the data used for model selection. To examine this, the final YOLOv11s model was evaluated at two levels: first on the held-out test set drawn from the same 12-camera pool as training and validation, and then on a separate unseen-camera benchmark built from three entirely unseen cameras.

On the held-out test set, the model achieved an overall mAP50 of 0.772 and an overall mAP50–95 of 0.607. These results remained reasonably close to the validation performance, although both were lower than the validation values of mAP50 of 0.857 and mAP50–95 of 0.661. The drop of 0.085 in mAP50 and 0.054 in mAP50–95 suggests that the validation improvements were real, but that performance still decreased when the model was applied to a fully separate split. In other words, the detector generalized well within the original dataset, but not without some loss.

The larger drop appeared on the cross-camera benchmark, where overall mAP50 fell to 0.658 and overall mAP50–95 fell to 0.488 (Table 17). Relative to the held-out test set, this corresponds to an additional decrease of 0.114 in mAP50 and 0.119 in mAP50–95. This result shows that the model transferred less effectively to unseen camera views than to new images from the original camera set [80], [81]. The imagery in this benchmark, shown in Figure 32, also presented more challenging visual conditions than the training set. The unseen cameras showed broader urban scenes, more background content, and many vehicles that appeared smaller because they were farther from the camera. Broadway and Houston was especially difficult, since it captured a wider street view with more surrounding buildings and more distant vehicles.

Table 21 shows that the class-level mAP50 differences from the test set to the cross-camera benchmark were not uniform. Passenger trucks were the most stable major class, while passenger cars, light commercial trucks, trucks, and especially buses were more sensitive to the unseen views. This pattern suggests that cross-camera generalization depended not just on object size, but on how consistently each class appeared across camera perspectives. Passenger trucks seem to have retained enough distinctive visual structure to transfer relatively well, whereas buses and light-duty classes were more disrupted by the new viewpoints and scene composition. The bus decline was especially notable, indicating that

Table 21: Comparison of held-out test set and cross-camera benchmark performance for the final YOLOv11s model. The change column reports the difference between the cross-camera benchmark and the held-out test set ( $\Delta = \text{cross-camera} - \text{held-out test}$ ).

| Metric                      | Test Set | Cross-Camera Benchmark | Change ( $\Delta$ ) |
|-----------------------------|----------|------------------------|---------------------|
| Overall mAP50               | 0.772    | 0.658                  | -0.114              |
| Overall mAP50–95            | 0.607    | 0.488                  | -0.119              |
| Passenger Car AP50          | 0.779    | 0.626                  | -0.153              |
| Passenger Truck AP50        | 0.868    | 0.761                  | -0.107              |
| Light Commercial Truck AP50 | 0.867    | 0.685                  | -0.182              |
| Truck AP50                  | 0.926    | 0.735                  | -0.191              |
| Bus AP50                    | 0.895    | 0.635                  | -0.260              |
| Motorcycle AP50             | 0.298    | 0.505                  | 0.207               |

strong performance on the original camera set did not automatically carry over to substantially different views. Motorcycle mAP50 increased on the cross-camera benchmark, but that result should be interpreted cautiously because it was based on only two labeled motorcycles, making it too small a sample to support a broader conclusion.

Taken together, these results show two levels of generalization. The model remained strong on the held-out test set, which indicates that the selected improvements were not limited to the validation split alone. However, performance dropped more noticeably on the unseen-camera benchmark, showing that transfer to new camera views is still a harder problem. For deployment, this suggests that the detector can still be used out of the box for broad traffic monitoring, but more accurate class-specific estimation would likely require adding labeled example images from each new location to the training set. In particular, brief site-specific fine-tuning would likely help the detector adapt to new scene layouts, viewing distances, and vehicle appearance at that camera.

## 6.6 Analysis of Traffic Count Agreement

The ATR comparison in Section 5.4 was used to test whether the detection pipeline could recover not only the daily pattern of traffic, but also the overall scale of vehicle activity. To evaluate this, three metrics were used: Pearson correlation, mean absolute error (MAE), and mean absolute percentage error (MAPE). In the context of this study, Pearson correlation measures how well the detector matched the shape of the hourly traffic pattern, regardless of total count magnitude. MAE measures the average hourly difference in raw

vehicle counts. MAPE measures the average hourly error as a percentage of the ATR count, which makes it easier to compare error across locations with different traffic volumes.

As shown in Table 19, the closest agreement was at Atlantic Avenue at Boerum Place. On March 18, the detector produced a daily total of 7,559 vehicles compared with an ATR total of 8,725, with a Pearson correlation of 0.844, an MAE of 82.67 vehicles per hour, and a MAPE of 22.96%. On March 19, agreement remained fairly strong, with 7,060 YOLO-derived vehicles versus 9,188 ATR vehicles, a Pearson correlation of 0.803, an MAE of 104.17, and a MAPE of 27.28%. These were the best results in the comparison, suggesting that at this location the detector captured both the hourly trend and the total volume reasonably well.

The other two locations showed a different pattern. At Pelham Parkway Westbound at Boston Road, Pearson correlation was consistently strong, ranging from 0.780 to 0.906, which indicates that the detector generally tracked the timing of higher- and lower-volume hours well. However, MAE ranged from 421.04 to 644.83 vehicles per hour, and MAPE ranged from 70.66% to 78.47%. This means that the model captured the daily traffic shape but substantially undercounted the true number of vehicles. Broadway and Houston showed the weakest and least consistent agreement overall. Pearson correlation ranged from 0.234 to 0.753, while MAPE remained high on all four days, from 67.22% to 69.08%. The worst single-day result occurred on March 16, when the detector counted 3,431 vehicles compared with 12,382 from the ATR, with a Pearson correlation of only 0.234. This indicates that at Broadway and Houston, the method sometimes failed to recover even the basic hourly pattern reliably.

These results show a clear difference between pattern agreement and count agreement. High Pearson correlation, as seen especially at Pelham Parkway, means the detector was often able to identify when traffic was relatively busy or quiet over the course of the day. High MAPE and MAE, however, show that the absolute count scale could still be far off. For this project, that distinction matters. If the goal is to estimate temporal traffic dynamics, such as peak hours or relative changes in flow, then strong correlation is useful. But if the goal is to estimate total vehicle activity for emissions modeling, then large magnitude errors remain a serious limitation.

Several factors likely contributed to this undercounting. First, the camera-based method only sampled one image every two minutes, so vehicles passing between snapshots were never observed. Second, the detector could only count vehicles that were visible within the camera's field of view, unlike ATR sensors that record traffic continuously across the

roadway. Occlusion likely also played a role, especially when larger vehicles blocked smaller ones. More generally, this comparison was necessarily approximate because the ATR counts came from 2025 while the camera images came from 2026. The matched dates were therefore the best available comparison, but they were not exact same-day ground truth. In addition, several of the matched days fell during the week of Saint Patrick’s Day, so location-specific events, celebrations, or traffic disruptions may have caused year-to-year differences that were unrelated to detector performance.

Overall, the ATR comparison suggests that the pipeline was more successful at recovering the relative hourly structure of traffic than at recovering exact vehicle totals. Atlantic Avenue showed that closer agreement was possible in a favorable view, while Pelham Parkway showed that strong temporal tracking could coexist with severe undercounting. Broadway and Houston demonstrated that in more difficult scenes, even the hourly pattern could break down. For practical use, this means the detector can provide useful traffic-pattern information, but absolute count estimates should be interpreted cautiously unless additional calibration is performed.

## 6.7 Implications for Public Health and Environmental Policy

The results of this study suggest that existing traffic-camera networks could support a more spatially detailed approach to traffic-related air-quality analysis. In New York City, traffic contributes about 14% of citywide  $PM_{2.5}$ , and prior work has shown that trucks and buses account for a particularly large share of traffic-attributable ambient  $PM_{2.5}$  in the city [7]. Within that context, the main contribution of this pipeline is not direct pollution measurement, but the ability to estimate vehicle composition at specific intersections using a MOVES-aligned taxonomy. That is potentially useful because fixed air-quality monitors provide only limited spatial coverage, whereas camera-based classification could help identify locations with persistently high proportions of buses, trucks, or other higher-emitting vehicle groups that are especially relevant for street-level exposure.

At the same time, the results also show that this approach should be treated as a screening and prioritization tool rather than a full replacement for conventional monitoring. The detector performed strongly on the held-out test set, but cross-camera performance declined on unseen views, and the ATR comparison showed that hourly traffic patterns were recovered more reliably than exact count magnitude. In practical terms, this means the pipeline is better suited to highlighting relative differences across locations, time

periods, or vehicle mixes than to producing perfectly calibrated traffic totals without local adjustment.

Even so, that kind of intersection-level screening could still be useful for public-health and planning decisions, such as identifying corridors where freight or bus activity is concentrated and where follow-up interventions or measurements may be warranted. More broadly, the benefit of this approach is that it can build on infrastructure already being expanded by NYC DOT, which has described plans to scale connected-vehicle and smart-intersection deployments to as many as 600 intersections. If paired with limited site-specific calibration, the pipeline could offer a relatively low-cost and privacy-preserving way to add transportation exposure intelligence to existing urban traffic systems.

## 7 Limitations

Despite the success of the final YOLOv11s pipeline, several limitations must be acknowledged regarding the data, the modeling approach, and the downstream application of the system.

### 7.1 Imaging and Dataset

First, the  $352 \times 240$  resolution of the 511NY and NYSDOT camera feeds places a clear limit on how much visual detail the detector can use. Many vehicles appear too small, too blurry, or too occluded to classify confidently, which is reflected in the dataset by the large number of `unknown_vehicle` annotations. This low resolution is helpful from a privacy standpoint, since it makes license plates and faces effectively unreadable, but it also makes the system less reliable for smaller or farther vehicles. In addition, the camera imagery itself is not always consistent. As shown in Figure 4 and Figure 8, some cameras occasionally produce degraded frames due to servicing or strong lighting effects. Nighttime glare (Figure 34a), direct sunlight and lens flare (Figure 34b), and image corruption (Figure 34c) can also reduce visibility and make vehicle boundaries harder to distinguish. These imaging inconsistencies introduce an additional source of noise that cannot be fully controlled during training.

Furthermore, although the training set contains 1,200 unique images, it was drawn from only 12 camera intersections, with more than half located in Manhattan and fewer



Figure 34: Examples of image-quality issues observed in the 511NY and NYSDOT traffic-camera feeds. These artifacts reduce visibility and make vehicle boundaries harder to distinguish during annotation and detection.

examples from other boroughs such as Brooklyn, the Bronx, and Queens. Camera viewpoint presented an additional source of limitation. A fixed camera view can be useful for deployment because it provides consistent scene geometry, but the cameras used in this study are Pan-Tilt-Zoom (PTZ) cameras whose orientation and zoom can change over time. As a result, images collected from a given location may not capture every view that the same camera could produce in practice. This could help explain why performance dropped on the unseen-camera benchmark, as the model learned the original camera views well but generalized less reliably to new scene layouts and viewing angles. Class imbalance was another important limitation. Some categories, especially **motorcycle**, were represented by very few examples in training. With only 48 labeled motorcycles, that class had the weakest test-set performance and was often visually confused with bicycles or other small two-wheeled objects.

## 7.2 Modeling and Taxonomy

The 6-class taxonomy was designed to support MOVES-based emissions analysis, but it is still a simplified representation of the full vehicle categories used in MOVES. The pipeline cannot recover the entire MOVES source-type hierarchy from imagery alone. More detailed distinctions, such as separating a transit bus from a private coach bus or distinguishing different truck types by use or weight class, depend on information that is not visible in a single low-resolution frame [51].

Additionally, the current pipeline does not capture the dynamic vehicle activity needed for full emissions modeling. Accurate MOVES-based emissions estimates depend not only on vehicle type, but also on operational variables such as speed and vehicle miles traveled

(VMT) [14]. Because this system processes isolated still frames, it cannot directly observe vehicle kinematics, stop-and-go behavior, or acceleration patterns. Although some of these quantities could be approximated using roadway speed limits or segment lengths, true operating conditions cannot be recovered from static snapshots alone.

In addition, the detector was evaluated only with spatial bounding-box metrics such as mAP50 and mAP50–95 on still images. These metrics measure detection and classification quality within individual frames, but they do not capture temporal consistency or full video-level understanding. More advanced models, such as large vision-language foundation models, may be able to reason about some ambiguous cases more flexibly, but they are not practical for this deployment setting. Running a large commercial model across hundreds of traffic cameras at frequent intervals would introduce substantial cost and latency. For that reason, efficient detectors such as YOLO remain the more realistic choice for scalable municipal monitoring, even if that comes with some loss of classification detail.

### 7.3 Downstream Counting and Deployment

A major limitation of the proposed system is that it operates on isolated still frames rather than continuous video. The current pipeline does not track vehicles across frames or assign persistent IDs over time, so it cannot determine whether a vehicle seen in one image is the same vehicle seen again later. It also cannot recover trajectories for vehicles that are briefly hidden by larger vehicles, surrounding infrastructure, or other occlusions. This limits the system’s ability to measure true vehicle flow in the same way as a continuous traffic sensor such as an automated traffic recorder (ATR).

The two-minute sampling interval also creates an additional mismatch between detections and real traffic counts. Vehicles that pass through the scene between snapshots are missed entirely, which helps explain the strong undercounting observed at faster-moving corridors such as Pelham Parkway. At the same time, vehicles that remain visible across multiple frames, such as parked cars, queued vehicles, or idling delivery trucks, may be counted more than once. As a result, the current approach is better suited to capturing broad traffic patterns than to recovering exact continuous vehicle volumes.

Overall, these limitations show that the current system is better suited to estimating vehicle composition in individual snapshots than to directly producing exact traffic volumes or full MOVES-ready emissions inputs. While the detector performs well as a frame-level classification tool, extending it to continuous counting and end-to-end emissions



estimation would require temporal tracking, count calibration, and additional operational data such as vehicle speed. The present pipeline should therefore be viewed as a practical first step toward camera-based traffic emissions monitoring, rather than a complete replacement for ATR infrastructure or full emissions modeling by itself.

## 8 Future Work

A natural next step is expanding dataset coverage and cross-camera generalization. The current detector was trained on imagery from only 12 intersections, and the cross-camera benchmark showed that performance dropped on unseen views. Expanding the dataset to include a wider range of camera locations, boroughs, lighting conditions, seasons, and viewing angles would make the system more robust for citywide deployment. Additional targeted annotation would also be especially valuable for underrepresented classes such as `motorcycle`, which had very few training examples and remained the least reliable class.

A second priority is improving how frame-level detections are converted into traffic counts. The ATR comparison showed that the current snapshot-based approach was better at recovering broad hourly traffic patterns than exact count magnitude. Future systems should therefore move beyond independent still-frame detection and incorporate temporal modeling. In particular, adding a lightweight multi-object tracking method would help prevent repeated counting of persistent vehicles between snapshots. A statistical calibration step that maps image-derived counts to ATR-style flow measurements would also help bridge the gap between detector output and true traffic volume.

A final direction is strengthening the connection between vehicle detection and emissions estimation. Although the current 6-class taxonomy was designed to align with MOVES, full MOVES-based emissions modeling also depends on operational variables such as vehicle speed and vehicle miles traveled. Integrating camera-based detections with external traffic data, roadway geometry, or speed estimates would make the pipeline more useful for downstream environmental analysis. Additional extensions could also explore predicting semantic attributes such as taxi or delivery status while preserving the core MOVES-aligned vehicle classes. Together, these improvements would move the pipeline from snapshot-based fleet characterization toward a more complete system for intersection-level traffic and emissions monitoring.

## 9 Conclusion

This thesis investigated whether existing New York City traffic-camera infrastructure could be used as a practical source of vehicle composition data for traffic and emissions analysis. To address this question, a low-resolution urban traffic dataset was constructed from 511NY and NYSDOT camera imagery, annotated using a MOVES-aligned 6-class vehicle taxonomy, and used to evaluate several modern object-detection architectures. Across the models tested, the YOLO family performed best for this setting, with the baseline YOLOv11s model achieving a mAP50 of 0.804 and a mAP50–95 of 0.615 on the validation set. After targeted refinement, the final selected YOLOv11s configuration reached a mAP50 of 0.857 and a mAP50–95 of 0.661 on the validation set, showing that careful data curation and hard negative mining were more effective in this case than simply changing to a more complex architecture or adding more untargeted data.

The results also showed that the design of the taxonomy and the treatment of ambiguous annotations were central to the success of the pipeline. By grouping visually similar vehicle body styles into broader MOVES-relevant classes and treating `unknown_vehicle` regions as ignore areas rather than a learnable class, the detector was able to focus on vehicle distinctions that were both visually reliable and meaningful for downstream environmental analysis. This made the final system a better match for the limitations of  $352 \times 240$  traffic imagery, where many vehicles are too distant, too small, or too occluded to support fine-grained labeling. In that sense, the strongest gains in this study came not only from model selection, but from aligning the dataset and class structure with the actual information present in the images.

Beyond validation performance, the experiments provided a clearer picture of where the detector generalized well and where important limitations remained. On the held-out test set, the final model achieved an overall mAP50 of 0.772 and a mAP50–95 of 0.607, indicating that the selected improvements transferred beyond the validation split. On the separate cross-camera benchmark of unseen locations, performance fell to a mAP50 of 0.658 and a mAP50–95 of 0.488, showing that transfer to new camera views was still more difficult. The comparison between real 2025 NYC ATR counts further showed that the pipeline was better at recovering the hourly pattern of traffic than exact count magnitude. For example, Atlantic Avenue at Boerum Place produced the closest count agreement, with Pearson correlations of 0.844 and 0.803 and MAPE values of 22.96% and 27.28%, while Pelham Parkway maintained strong hourly-profile agreement, with correlations up to 0.906, but still severely undercounted total vehicles. These findings suggest that the current

system is already useful for broad fleet characterization and traffic pattern analysis, but that exact count recovery and camera-to-camera transfer remain open challenges.

The results support the overall feasibility of using public traffic camera imagery for intersection-level vehicle classification in support of transportation and environmental analysis. At the same time, the work also shows that this approach is best understood as a first stage rather than a complete end-to-end emissions-monitoring solution. The current pipeline can estimate vehicle composition within snapshots and capture broad temporal traffic structure, but extending it to fully calibrated traffic volumes and MOVES-ready emissions inputs will require broader training data, stronger cross-camera adaptation, temporal tracking, and additional operational information such as vehicle speed. Even with these limitations, the study demonstrates that existing traffic-camera networks can provide useful transportation intelligence without requiring new physical sensing infrastructure. In that sense, this work establishes a practical foundation for future camera-based traffic and emissions monitoring in urban environments.

# Appendix

## A Transformer Training Augmentation Pipeline

The training augmentations used in the transformer-based detector experiments were implemented with the Albumentations Python library. Because these transformations directly modify both the input image and its associated bounding boxes, Albumentations was used to ensure that geometric and photometric augmentations remained consistent with the object detection annotations.

The heavier augmentation pipeline consisted of:

- Horizontal flipping with probability 0.5.
- Random safe cropping and resizing to  $460 \times 768$  with probability 0.25.
- Perspective transformation with probability 0.1.
- One of two color perturbations with overall probability 0.9:
  - Hue, saturation, and value shifting.
  - Random brightness and contrast adjustment.
- Gaussian blur with probability 0.1.

In Python, this was implemented as:

```
import albumentations as A

# Heavier augmentation pipeline
def get_train_transforms():
    return A.Compose([
        A.HorizontalFlip(p=0.5),
        A.RandomSizedBBBoxSafeCrop(height=460, width=768, p=0.25),
        A.Perspective(p=0.1),
        A.OneOf([
            A.HueSaturationValue(
                hue_shift_limit=20,
```

```

        sat_shift_limit=30,
        val_shift_limit=20,
        p=0.9
    ),
    A.RandomBrightnessContrast(
        brightness_limit=0.2,
        contrast_limit=0.2,
        p=0.9
    ),
], p=0.9),
A.Blur(blur_limit=3, p=0.1),
], bbox_params=A.BboxParams(
    format="pascal_voc",
    label_fields=["labels"],
    min_visibility=0.1
))

```

The lighter augmentation pipeline consisted of:

- Horizontal flipping with probability 0.5.
- Random brightness and contrast adjustment with brightness limit 0.12, contrast limit 0.12, and probability 0.3.

In Python, this was implemented as:

```

import albumentations as A

# Lighter augmentation pipeline
def get_train_transforms():
    return A.Compose([
        A.HorizontalFlip(p=0.5),
        A.RandomBrightnessContrast(
            brightness_limit=0.12,
            contrast_limit=0.12,
            p=0.3
        )
    ])

```

```

    ),
    ], bbox_params=A.BboxParams(
        format="pascal_voc",
        label_fields=["labels"],
        min_visibility=0.1
    ))

```

In both cases, bounding boxes were processed using Albumentations with Pascal VOC box format. The associated class labels were tracked through the `label_fields` argument, and boxes with visibility below 0.1 after transformation were discarded. As discussed in Section 5.2.2, the heavier augmentation pipeline was used in the initial Deformable DETR experiment, while the lighter pipeline was used in the later transformer-based experiments. The lighter pipeline produced stronger validation performance and was therefore used for most of the transformer results reported in Section 5.2.2.

## References

- [1] World Health Organization. “Air pollution.” (2026), [Online]. Available: <https://www.who.int/health-topics/air-pollution> (visited on 02/13/2026).
- [2] Health Effects Institute, “State of global air 2025: A report on air pollution and its role in the world’s leading causes of death,” Health Effects Institute, Boston, MA, 2025. [Online]. Available: [www.stateofglobalair.org](http://www.stateofglobalair.org).
- [3] IPCC, “Climate change 2022: Mitigation of climate change. contribution of working group III to the sixth assessment report of the intergovernmental panel on climate change,” Intergovernmental Panel on Climate Change, Geneva, Switzerland, 2022. DOI: 10.1017/9781009157926. [Online]. Available: <https://www.ipcc.ch/report/ar6/wg3/>.
- [4] F. Creutzig, P. Jochem, O. Y. Edelenbosch, *et al.*, “Transport: A roadblock to climate change mitigation?” *Science*, vol. 350, no. 6263, pp. 911–912, Nov. 20, 2015, ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.aac8033. [Online]. Available: <https://www.science.org/doi/10.1126/science.aac8033> (visited on 02/13/2026).

- [5] E. Uherek, T. Halenka, J. Borken-Kleefeld, *et al.*, “Transport impacts on atmosphere and climate: Land transport,” *Atmospheric Environment*, Transport Impacts on Atmosphere and Climate: The ATTICA Assessment Report, vol. 44, no. 37, pp. 4772–4816, Dec. 1, 2010, ISSN: 1352-2310. DOI: 10.1016/j.atmosenv.2010.01.002. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1352231010000099> (visited on 02/13/2026).
- [6] New York City Department of City Planning. “Population,” Population - Department of City Planning. (Jul. 2024), [Online]. Available: <https://www.nyc.gov/content/planning/pages/planning/population> (visited on 02/13/2026).
- [7] New York City Department of Health {and} Mental Hygiene. “Traffic and PM2.5 air pollution,” Environment & Health Data Portal. (2026), [Online]. Available: <https://a816-dohbsp.nyc.gov/IndicatorPublic/data-stories/traffic-and-air-pollution/> (visited on 02/13/2026).
- [8] Y. Wang, H. Wang, B. Zhang, *et al.*, “High-resolution mapping of on-road vehicle emissions with real-time traffic datasets based on big data,” *Atmospheric Chemistry and Physics*, vol. 25, no. 11, pp. 5537–5555, Jun. 4, 2025, ISSN: 1680-7316. DOI: 10.5194/acp-25-5537-2025. [Online]. Available: <https://acp.copernicus.org/articles/25/5537/2025/> (visited on 12/29/2025).
- [9] Federal Highway Administration. “Traffic monitoring: A guidebook — 4. traffic monitoring equipment capabilities & limitations,” Center for Local Aid Support (CTIP Publications) — Federal Highway Administration. (2010), [Online]. Available: [https://www.fhwa.dot.gov/clas/ctip/traffic\\_monitoring\\_guidebook/ch\\_4.aspx](https://www.fhwa.dot.gov/clas/ctip/traffic_monitoring_guidebook/ch_4.aspx) (visited on 03/06/2026).
- [10] The City of New York, Department of Transportation. “Automated traffic volume counts — NYC open data.” (2026), [Online]. Available: [https://data.cityofnewyork.us/Transportation/Automated-Traffic-Volume-Counts/7ym2-wayt/about\\_data](https://data.cityofnewyork.us/Transportation/Automated-Traffic-Volume-Counts/7ym2-wayt/about_data) (visited on 03/07/2026).
- [11] New York State Department of Transportation, *C038145, transportation management system and operations (TMSO) support software & c038150 TSMO control software project management support and QA/QC*, Albany, NY, Mar. 13, 2025. [Online]. Available: [https://www.dot.ny.gov/portal/pls/portal/MEXIS\\_APP.BC\\_CONSULTING\\_NONAE\\_ADMIN.VIEWFILE?p\\_file\\_id=47766](https://www.dot.ny.gov/portal/pls/portal/MEXIS_APP.BC_CONSULTING_NONAE_ADMIN.VIEWFILE?p_file_id=47766).



- [12] New York State Department of Transportation. “Real time traffic information.” (2026), [Online]. Available: <https://webcams.nyctmc.org/cameras-list> (visited on 03/02/2026).
- [13] New York State Department of Transportation. “511ny - API documentation,” API Documentation. (2009), [Online]. Available: <https://511ny.org/developers/511ny.org> (visited on 03/02/2026).
- [14] U.S. Environmental Protection Agency, “MOVES3 technical guidance: Using MOVES to prepare emission inventories for state implementation plans and transportation conformity,” Office of Transportation and Air Quality, U.S. Environmental Protection Agency, EPA-420-B-20-052, Nov. 2020. [Online]. Available: <https://www.epa.gov/sites/default/files/2020-11/documents/420b20052.pdf> (visited on 03/02/2026).
- [15] U.S. Environmental Protection Agency, “MOVES5 technical guidance: Using MOVES to prepare emission inventories for state implementation plans and transportation conformity,” Office of Transportation and Air Quality, U.S. Environmental Protection Agency, Ann Arbor, MI, EPA-420-B-24-043, Nov. 2024. [Online]. Available: <https://nepis.epa.gov/Exe/ZyPDF.cgi?Dockey=P101CTM3.pdf> (visited on 03/07/2026).
- [16] U.S. Environmental Protection Agency, “Overview of EPA’s MOtor vehicle emission simulator (MOVES5),” Office of Transportation and Air Quality, U.S. Environmental Protection Agency, EPA-420-R-25-005, Feb. 2025. [Online]. Available: <https://nepis.epa.gov/Exe/ZyPDF.cgi?Dockey=P101DQ9V.pdf> (visited on 03/07/2026).
- [17] H. Hill, “Producing a NYMTC GHG inventory,” September 11th Memorial Program for Transportation Planning. [Online]. Available: [https://www.nymtc.org/portals/0/pdf/presentations/inventory\\_911.pdf](https://www.nymtc.org/portals/0/pdf/presentations/inventory_911.pdf) (visited on 03/07/2026).
- [18] California Air Resources Board. “Brake & tire wear emissions,” California Air Resources Board. (2021), [Online]. Available: <https://ww2.arb.ca.gov/resources/documents/brake-tire-wear-emissions> (visited on 03/07/2026).

- [19] Orange County Transportation Council (OCTC), “Final transportation/air quality conformity determination for the orange county portion of the NY-NJ-CT pm2.5 maintenance area,” Orange County Transportation Council (OCTC), Goshen, NY, Mar. 2, 2021. [Online]. Available: <https://www.orangecountygov.com/Archive/ViewFile/Item/1212> (visited on 03/07/2026).
- [20] M. Tayarani, A. Baghestani, M. Allahviranloo, and H. O. Gao, “Spatial/temporal variability in transportation emissions and air quality in nyc cordon pricing,” *Transportation Research Part D: Transport and Environment*, vol. 89, p. 102620, 2020, ISSN: 1361-9209. DOI: <https://doi.org/10.1016/j.trd.2020.102620>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1361920920308051>.
- [21] K. Zhang and S. Batterman, “Air pollution and health risks due to vehicle traffic,” *Science of The Total Environment*, vol. 450-451, pp. 307–316, 2013, ISSN: 0048-9697. DOI: <https://doi.org/10.1016/j.scitotenv.2013.01.074>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0048969713001290>.
- [22] New York City Department of Transportation. “NYC DOT - infrastructure - traffic signals.” (2026), [Online]. Available: <https://www.nyc.gov/html/dot/html/infrastructure/signals.shtml> (visited on 03/02/2026).
- [23] *Induction loop*, in *Wikipedia*, Page Version ID: 1295587875, Jun. 14, 2025. [Online]. Available: [https://en.wikipedia.org/w/index.php?title=Induction\\_loop&oldid=1295587875](https://en.wikipedia.org/w/index.php?title=Induction_loop&oldid=1295587875) (visited on 03/07/2026).
- [24] Mid-Atlantic Entry Systems. “Vehicle detection loops: How they work and why they fail.” (2026), [Online]. Available: <https://www.midatlanticentry.com/article/vehicle-detection-loops-how-they-work-and-why-they-fail/> (visited on 03/06/2026).
- [25] Federal Highway Administration. “Chapter 4 - in-roadway sensor technologies - vehicle detection and surveillance technologies,” Office of Highway Policy Information - Policy — Federal Highway Administration. (Nov. 7, 2014), [Online]. Available: <https://www.fhwa.dot.gov/policyinformation/pubs/vdstits2007/04.cfm> (visited on 03/06/2026).

- [26] MetroCount. “RoadPod VT4 — 4-tube vehicle counter.” (2025), [Online]. Available: <https://www.metrocount.com/traffic-counters-classifiers/roadpod-vt4> (visited on 03/07/2026).
- [27] New York City Mayor’s Office of Environmental Coordination, “CEQR technical manual, december 2025 edition: Appendix: Transportation,” New York City Mayor’s Office of Environmental Coordination, New York, NY, Dec. 2025. [Online]. Available: [https://www.nyc.gov/assets/oec/technical-manual/2025\\_ceqr\\_tm\\_appendix\\_transportation.pdf](https://www.nyc.gov/assets/oec/technical-manual/2025_ceqr_tm_appendix_transportation.pdf) (visited on 03/06/2026).
- [28] New York Metropolitan Transportation Council, “Hub bound travel data report 2024,” New York Metropolitan Transportation Council (NYMTC), New York, NY, 2026. [Online]. Available: [https://www.nymtc.org/Portals/0/Pdf/Hub%20Bound/2025%20Hub%20Bound/2024%20Hub%20Bound%20Report%20\(2\).pdf?ver=K0jQ6a7ZBmPSZaM8WYDkAg%3d%3d](https://www.nymtc.org/Portals/0/Pdf/Hub%20Bound/2025%20Hub%20Bound/2024%20Hub%20Bound%20Report%20(2).pdf?ver=K0jQ6a7ZBmPSZaM8WYDkAg%3d%3d) (visited on 03/06/2026).
- [29] New York City Department of Transportation. “NYC DOT to test new technology to improve street safety by better measuring and analyzing transportation uses on NYC streets.” (Apr. 12, 2023), [Online]. Available: <https://www.nyc.gov/html/dot/html/pr2023/dot-new-technology-improve-safety.shtml> (visited on 03/07/2026).
- [30] VivaCity. “Viva - smarter, safer, sustainable cities,” Viva North America. (2026), [Online]. Available: <https://vivacitylabs.com/north-america/> (visited on 03/07/2026).
- [31] C2SMART. “Manhattan congestion tracker dashboard\_viz1 - c2smart home.” (Apr. 1, 2025), [Online]. Available: [https://c2smart.engineering.nyu.edu/manhattan-congestion-tracker-dashboard\\_viz1/](https://c2smart.engineering.nyu.edu/manhattan-congestion-tracker-dashboard_viz1/) (visited on 03/07/2026).
- [32] M. K. Turkcan, J. Tavori, J. Ghaderi, G. Zussman, Z. Kostic, and A. Smyth, *A vision-based analysis of congestion pricing in new york city*, 2026. arXiv: 2602.03015 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2602.03015>.
- [33] J. Krause, J. Deng, M. Stark, and L. Fei-Fei, “Collecting a large-scale dataset of fine-grained cars,” 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16632981>.

- [34] Z. Dong, Y. Wu, M. Pei, and Y. Jia, "Vehicle type classification using a semisupervised convolutional neural network," *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 4, pp. 2247–2256, Aug. 2015, ISSN: 1558-0016. DOI: 10.1109/TITS.2015.2402438. [Online]. Available: <https://ieeexplore.ieee.org/document/7055873> (visited on 03/02/2026).
- [35] Office of the Governor. "Governor hochul signs legislation to expand red light camera programs and protect new yorkers on the road," Governor Kathy Hochul - NY.gov. (Oct. 23, 2024), [Online]. Available: <https://www.governor.ny.gov/news/governor-hochul-signs-legislation-expand-red-light-camera-programs-and-protect-new-yorkers> (visited on 03/02/2026).
- [36] New York City Department of Transportation. "NYC DOT - data feeds." (2026), [Online]. Available: <https://www.nyc.gov/html/dot/html/about/datafeeds.shtml> (visited on 03/02/2026).
- [37] New York State Department of Transportation. "About us - 511ny rideshare - 511ny.org," 511NY Rideshare. (2009), [Online]. Available: <https://511nyrideshare.org/about-us> (visited on 03/02/2026).
- [38] New York State Department of Transportation, "Item 683.03010005 – networking equipment," NYSDOT Office of Engineering, 683.03010005, Jun. 2024. [Online]. Available: <https://www.dot.ny.gov/spec-repository-us/683.03010005.pdf>.
- [39] New York City Department of Transportation, *Real time traffic speed data – metadata*, Jul. 20, 2010. [Online]. Available: <https://www.nyc.gov/html/dot/downloads/pdf/metadata-trafficspeeds.pdf> (visited on 03/02/2026).
- [40] New York State Department of Transportation, "Item 683.03120011 – managed ethernet switch – layer 2," NYSDOT Office of Engineering, 683.03120011, Oct. 2014. [Online]. Available: <https://www.dot.ny.gov/spec-repository-us/683.03120011.pdf>.
- [41] B&H Photo Video, *Pelco ES31c22-2w esprit positioning system (24 VAC) product image*, 2026. [Online]. Available: [https://www.bhphotovideo.com/c/product/500013-REG/Pelco\\_ES31C222W\\_ES31C22\\_2W\\_Esprit\\_Positioning.html](https://www.bhphotovideo.com/c/product/500013-REG/Pelco_ES31C222W_ES31C22_2W_Esprit_Positioning.html) (visited on 03/02/2026).

- [42] froehlich5220, *PELCO CM9760-CXTA 16-PORT COAXITRON TRANSLATOR 9760 SURVEILLANCE VIDEO SYSTEM*, 2026. [Online]. Available: <https://www.ebay.com/itm/177535176993> (visited on 03/02/2026).
- [43] Axis Communications, *AXIS q1614-e network camera product image*, 2026. [Online]. Available: <https://www.axis.com/products/axis-q1614-e/support> (visited on 03/02/2026).
- [44] New York City Department of Transportation. “NYC DOT - truck routing,” Truck Routing. (2026), [Online]. Available: <https://www.nyc.gov/html/dot/html/motorist/truckrouting.shtml> (visited on 03/03/2026).
- [45] New York City Department of Transportation, *Delivering new york: A smart truck management plan for new york city*, New York, NY, May 13, 2021. [Online]. Available: <https://www.nyc.gov/html/dot/downloads/pdf/smart-truck-management-plan.pdf> (visited on 03/02/2026).
- [46] New York City Department of Transportation. “Real time traffic information - subscribers,” Subscribers. (2026), [Online]. Available: <https://webcams.nyctmc.org/subscribers> (visited on 03/02/2026).
- [47] CVAT.ai Corporation. “Computer vision annotation tool (cvat),” CVAT.ai. (2024), [Online]. Available: <https://www.cvat.ai> (visited on 03/11/2026).
- [48] Federal Highway Administration. “Traffic monitoring guide (2013): Appendix c. vehicle types,” Office of Highway Policy Information (OHPI) – Traffic Monitoring Guide. (Nov. 7, 2014), [Online]. Available: [https://www.fhwa.dot.gov/policyinformation/tmguidetmg\\_2013/vehicle-types.cfm](https://www.fhwa.dot.gov/policyinformation/tmguidetmg_2013/vehicle-types.cfm) (visited on 03/11/2026).
- [49] U.S. Environmental Protection Agency. “How does MOVES define light duty trucks?” (Feb. 22, 2019), [Online]. Available: <https://www.epa.gov/moves/how-does-moves-define-light-duty-trucks> (visited on 03/11/2026).
- [50] U.S. Department of Energy. “Alternative fuels data center: Maps and data - vehicle weight classes and categories.” (2026), [Online]. Available: <https://afdc.energy.gov/data/10380> (visited on 03/11/2026).

- [51] U.S. Environmental Protection Agency, “Population and activity of onroad vehicles in MOVES3,” Office of Transportation and Air Quality, U.S. Environmental Protection Agency, EPA-420-R-20-023, Nov. 2020. [Online]. Available: <https://www.epa.gov/sites/default/files/2020-11/documents/420r20023.pdf> (visited on 03/11/2026).
- [52] CVAT.ai Corporation. “Labeling guidelines: How to keep annotations consistent — CVAT academy,” CVAT.ai. (2023), [Online]. Available: <https://www.cvat.ai/academy/labeling-guidelines> (visited on 03/11/2026).
- [53] CVAT.ai Corporation. “Attributes in CVAT: Add object properties — CVAT academy,” CVAT.ai. (2023), [Online]. Available: <https://www.cvat.ai/academy/label-attributes> (visited on 03/11/2026).
- [54] CVAT.ai Corporation. “Annotation quality: What “good labels” look like — CVAT academy,” CVAT.ai. (2023), [Online]. Available: <https://www.cvat.ai/academy/annotation-quality-principles> (visited on 03/12/2026).
- [55] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, *You only look once: Unified, real-time object detection*, 2016. arXiv: 1506.02640 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1506.02640>.
- [56] GeeksforGeeks. “Introduction to convolution neural network,” GeeksforGeeks. (Aug. 21, 2017), [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/> (visited on 03/19/2026).
- [57] M. Islam. “Pooling layers in convolutional neural networks (CNNs) – a beginner’s guide.” (Mar. 8, 2025), [Online]. Available: <https://www.linkedin.com/pulse/pooling-layers-convolutional-neural-networks-cnns-beginners-islam-i76ac> (visited on 03/19/2026).
- [58] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [59] K. O’Shea and R. Nash, *An introduction to convolutional neural networks*, 2015. arXiv: 1511.08458 [cs.NE]. [Online]. Available: <https://arxiv.org/abs/1511.08458>.
- [60] R. Khanam and M. Hussain, *Yolov11: An overview of the key architectural enhancements*, 2024. arXiv: 2410.17725 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2410.17725>.

- [61] G. Jocher and J. Qiu, *Ultralytics yolo11*, version 11.0.0, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [62] T.-Y. Lin, M. Maire, S. Belongie, *et al.*, “Microsoft COCO: Common objects in context,” arXiv, Feb. 21, 2015. DOI: 10.48550/arXiv.1405.0312. arXiv: 1405.0312. [Online]. Available: <http://arxiv.org/abs/1405.0312> (visited on 03/01/2026).
- [63] Ultralytics. “COCO dataset.” (2024), [Online]. Available: <https://docs.ultralytics.com/datasets/detect/coco/> (visited on 04/06/2026).
- [64] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics yolov8*, version 8.0.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [65] Ultralytics. “YOLOv8 vs YOLO11: A comprehensive technical comparison of real-time vision models.” (2025), [Online]. Available: <https://docs.ultralytics.com/compare/yolov8-vs-yolo11/> (visited on 04/06/2026).
- [66] G. Jocher and J. Qiu, *Ultralytics yolo26*, version 26.0.0, 2026. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [67] S. Ren, K. He, R. Girshick, and J. Sun, *Faster r-cnn: Towards real-time object detection with region proposal networks*, 2016. arXiv: 1506.01497 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1506.01497>.
- [68] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, *Feature pyramid networks for object detection*, 2017. arXiv: 1612.03144 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1612.03144>.
- [69] T. maintainers and contributors, *Torchvision: Pytorch’s computer vision library*, <https://github.com/pytorch/vision>, 2016.
- [70] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, *End-to-end object detection with transformers*, 2020. arXiv: 2005.12872 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2005.12872>.
- [71] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, *Deformable detr: Deformable transformers for end-to-end object detection*, 2021. arXiv: 2010.04159 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2010.04159>.
- [72] Y. Zhao, W. Lv, S. Xu, *et al.*, *Detrs beat yolos on real-time object detection*, 2024. arXiv: 2304.08069 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2304.08069>.

- [73] W. Lv, Y. Zhao, Q. Chang, K. Huang, G. Wang, and Y. Liu, *Rt-detr2: Improved baseline with bag-of-freebies for real-time detection transformer*, 2024. arXiv: 2407.17140 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2407.17140>.
- [74] Ultralytics. “Model validation with ultralytics YOLO.” (2024), [Online]. Available: <https://docs.ultralytics.com/modes/val/> (visited on 04/06/2026).
- [75] P. Zippenfenig, *Open-meteo.com weather API*, Dec. 31, 2024. DOI: 10.5281/ZENODO.7970649. [Online]. Available: <https://zenodo.org/doi/10.5281/zenodo.7970649> (visited on 04/09/2026).
- [76] Ultralytics. “RTDETRv2 vs. YOLO11: A deep dive into real-time object detection architectures.” (2025), [Online]. Available: <https://docs.ultralytics.com/compare/rtdetr-vs-yolo11/> (visited on 04/10/2026).
- [77] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, *Learning from noisy labels with deep neural networks: A survey*, 2022. arXiv: 2007.08199 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2007.08199>.
- [78] B. Feuer, J. Xu, N. Cohen, P. Yubeaton, G. Mittal, and C. Hegde, *Select: A large-scale benchmark of data curation strategies for image classification*, 2024. arXiv: 2410.05057 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2410.05057>.
- [79] S. Goyal, P. Maini, Z. C. Lipton, A. Raghunathan, and J. Z. Kolter, *Scaling laws for data filtering – data curation cannot be compute agnostic*, 2024. arXiv: 2404.07177 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2404.07177>.
- [80] H.-K. Hsu, C.-H. Yao, Y.-H. Tsai, *et al.*, *Progressive domain adaptation for object detection*, 2019. arXiv: 1910.11319 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1910.11319>.
- [81] V. M. Patel, R. Gopalan, R. Li, and R. Chellappa, “Visual domain adaptation: A survey of recent advances,” *IEEE Signal Processing Magazine*, vol. 32, no. 3, pp. 53–69, 2015. DOI: 10.1109/MSP.2014.2347059.